



Continual learning for adaptive IoT network intrusion detection via domain-incremental learning methods

Oliver Delgado^a, Hyunjae Kang^{a,*}, Ulysses Lam^a, Jung Taek Seo^b, Dan Dongseong Kim^a

^a School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, QLD 4072, Australia

^b Department of Smart Security, Gachon University, Seongnam-si, Gyeonggi-do 13120, Republic of Korea

HIGHLIGHTS

- Continual learning is evaluated for IoT NIDS under domain-incremental settings.
- Replay-based methods effectively mitigate catastrophic forgetting across stages.
- Experience replay sustains high F1-scores with a small memory budget.
- Continual learning outperforms anomaly-based NIDS with limited training samples.
- Findings guide adaptive and resource-efficient IoT intrusion detection.

ARTICLE INFO

Keywords:

Continual learning
Catastrophic forgetting
IoT security
Network intrusion detection

ABSTRACT

The proliferation of Internet of Things (IoT) devices increases exposure to network intrusions and motivates the development of Network Intrusion Detection Systems (NIDS) that can adapt to newly emerging attacks. However, models retrained on new threats often suffer from catastrophic forgetting, resulting in degraded performance on previously observed attacks. This paper evaluates continual learning (CL) strategies for neural-network-based NIDS on three IoT/network intrusion datasets (UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017) under an incremental training protocol that sequentially introduces new attack types over nine stages while testing on a cumulative set of previously encountered attacks. We compare eight CL strategies spanning weight inheritance (WI), regularization (EWC, LwF, SI), replay (ER, A-GEM, DER++), and their WI-based combination (EWC + ER) across MLP, LSTM, and Transformer architectures, under both balanced and unbalanced protocols. Across the three datasets, replay-based strategies retain final-stage macro-F1 of 0.92–0.98 with average forgetting below 0.05, whereas regularization-only and naive sequential strategies suffer macro-F1 drops of 0.30–0.55 and forgetting rates above 0.32. Replay's resource footprint is modest (peak CL state below 0.4 MB on MLP at a 100-sample-per-class buffer; training time within 1.5× of weight inheritance), and the same ranking holds across the three deep architectures and the unbalanced regime when measured by PR-AUC and false positive rate at high recall. A comparison against three unsupervised anomaly detectors (KitNET, Isolation Forest, and an autoencoder) further indicates that supervised continual learning is the more effective choice when labeled samples of new attack classes are available. The results confirm that memory-based mechanisms are central to sustaining detection performance in domain-incremental IoT NIDS scenarios.

1. Introduction

The rapid expansion of the Internet of Things (IoT) has reshaped the digital landscape, embedding billions of interconnected devices into industrial, commercial, and critical infrastructure fabrics [1]. While this ubiquitous connectivity drives operational efficiency, it also introduces

a broader attack surface. IoT networks are inherently dynamic, characterized by heterogeneous device populations and fluid traffic patterns, creating an environment for evolving cyber-threats. Consequently, securing these ecosystems necessitates robust Network Intrusion Detection Systems (NIDS) capable of identifying malicious activity in real-time.

* Corresponding author.

Email addresses: o.delgado@student.uq.edu.au (O. Delgado), hyunjae.kang@uq.edu.au (H. Kang), u.lam@uq.edu.au (U. Lam), seojt@gachon.ac.kr (J.T. Seo), dan.kim@uq.edu.au (D.D. Kim).

<https://doi.org/10.1016/j.asoc.2026.116022>

Received 15 January 2026; Received in revised form 3 July 2026; Accepted 20 July 2026

Available online 21 July 2026

1568-4946/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

A critical limitation of current NIDS deployment models is their reliance on static batch learning. Conventional machine learning classifiers are typically trained offline on fixed historical datasets. When novel attack vectors emerge, a phenomenon known as *concept drift*, these static models rapidly become obsolete, failing to generalize to new threat distributions [2,3]. Typical remediation involves retraining the model from scratch using a cumulative dataset of historical and emerging patterns. However, in IoT environments constrained by limited computational resources, energy budgets, and bandwidth, frequent full-scale retraining is computationally prohibitive and operationally impractical.

Continual Learning (CL) offers a resource-efficient alternative by allowing models to update incrementally from data streams. However, standard neural networks suffer from *catastrophic forgetting* in this scenario; optimizing parameters for new tasks often destructively interferes with weights optimized for previous tasks, leading to a substantial decline in detection performance on historical threats [4]. This necessitates balancing the *plasticity-stability dilemma*, where the security system must remain plastic enough to integrate new attack patterns while being stable enough to retain protection against known attacks.

To rigorously evaluate this dilemma within a security context, we frame this study as Domain-Incremental Learning (DIL) [5,6]. Unlike Class-Incremental Learning (CIL), where the objective is to expand the label space to distinguish specific attack types, DIL requires the model to maintain a consistent decision boundary for the binary task (“Normal” vs. “Attack”) while the distribution of the “Attack” domain shifts sequentially. This formulation reflects the operational reality of an NIDS, which must continuously detect deviations regardless of the specific attack type. Furthermore, this distinction is important when interpreting regularization-based CL [4]. In IoT traffic, newly introduced attacks can induce substantial shifts in feature distributions, which may reduce parameter reuse across stages and limit the effectiveness of parameter-regularization methods such as Elastic Weight Consolidation.

While CL has been extensively studied in computer vision [7], its application to NIDS remains relatively underexplored. This study addresses this gap by empirically evaluating a broad set of CL strategies for adaptive intrusion detection across diverse network environments. Using three intrusion datasets that span IoT and enterprise traffic—UQ-IoT-IDS-2021 [8], Kitsune [9], and CIC-IDS-2017 [10]—we simulate the operational lifecycle of an NIDS encountering a stream of evolving threats. We systematically evaluate eight CL strategies organized by family: *Weight Inheritance* (WI) as a naive fine-tuning baseline; the regularization-based methods *Elastic Weight Consolidation* (EWC) [4], *Synaptic Intelligence* (SI) [11], and *Learning without Forgetting* (LwF) [12]; the replay-based methods *Experience Replay* (ER) [13], *Averaged Gradient Episodic Memory* (A-GEM) [14], and *Dark Experience Replay* + + (DER + +) [15]; and the combination EWC + ER, with joint training as an upper bound. Each strategy is evaluated on three neural architectures (MLP, LSTM, Transformer) under both balanced and unbalanced dataset regimes, and is further compared against classical Random Forest and SVM baselines as well as three unsupervised anomaly detectors (KitNET [9], an Isolation Forest, and an autoencoder). These findings suggest practical guidance for designing adaptive long-term security NIDS for IoT networks.

The contributions of our study are as follows:

- We design a CL evaluation framework for NIDS by formulating an incremental training and cumulative testing protocol that reflects how an NIDS is updated as new attack types emerge over time. The framework is instantiated through a uniform pipeline—covering feature handling, balanced and unbalanced sampling, and a fixed binary label space—enabling a systematic assessment of various CL strategies across heterogeneous IoT and enterprise datasets.
- We conduct experiments on three real-world intrusion datasets (UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017) using three deep architectures (MLP, LSTM, Transformer), evaluated under both balanced and unbalanced regimes against classical baselines (Random

Forest, SVM) and three unsupervised anomaly detectors (KitNET, Isolation Forest, autoencoder). We provide a quantitative performance analysis using a comprehensive metric set—macro-F1, the average forgetting rate F , PR-AUC, FPR at recall ≥ 0.95 , and per-attack F1—together with training-time overhead and peak CL-state memory to characterize deployment cost.

- We benchmark eight CL strategies that span the major families—weight inheritance (WI), regularization (EWC, SI, LwF), replay (ER, A-GEM, DER + +), and a combination (EWC + ER)—against a joint-training upper bound under nine sequential task scenarios that reflect time-evolving attack distributions. We further conduct sensitivity analyses on the regularization strength λ , the replay buffer size, and the task order to characterize the robustness of each strategy.

The rest of this paper is organized as follows. Section 2 reviews related work in CL and IoT NIDS. Section 3 details our proposed framework and the specific CL strategies employed. Section 4 describes the experimental setup, including the dataset and evaluation protocol. Section 5 presents the experimental results. Section 6 discusses the implications of our findings, and finally Section 7 concludes this paper.

2. Related work

2.1. Continual learning in network security

CL addresses the challenge of learning from a stream of data without forgetting previously acquired knowledge. CL strategies are broadly categorized into regularization, replay, and architecture-based methods [6], with recent adaptations extending to specific challenges such as adversarial defense [16]. In the context of NIDS, this requires models to adapt to new attack patterns without degrading detection performance on historical threats [4,17–19].

Within this framework, three task formulations are commonly distinguished [5,6]: *class-incremental learning* (CIL), where the label space expands over time and the model must distinguish an ever-growing set of classes; *task-incremental learning* (TIL), which additionally provides task identifiers at inference; and *domain-incremental learning* (DIL), where the label space remains fixed while the input distribution shifts across stages. The choice of formulation determines both the dominant source of forgetting and the appropriate evaluation protocol, and is particularly consequential for NIDS—where the operational detection objective (*Normal* vs. *Attack*) is fixed throughout deployment but the attack-side distribution evolves as new threats emerge.

2.1.1. Regularization-based approaches

Regularization-based approaches mitigate forgetting by constraining updates to weights deemed important for previously seen tasks. The most widely studied member of this family is Elastic Weight Consolidation (EWC) [4], which estimates parameter importance via a Fisher Information matrix at the end of each stage and penalizes deviations from the consolidated values during subsequent training. Synaptic Intelligence (SI) [11] estimates importance *online* by accumulating each parameter’s contribution to loss reduction along the training trajectory, removing the need for a separate post-stage Fisher pass. Learning without Forgetting (LwF) [12], in contrast, regularizes at the function level rather than the parameter level: a frozen snapshot of the previous model produces soft targets that are matched via a knowledge-distillation term during training on new data.

In intrusion detection, regularization-based methods are intuitively appealing because they preserve historical attack-related parameter structure without retaining raw samples. Recent work has explored hybrid extensions such as Elastic Variational Continual Learning (EVCL) [20], which combines EWC with Variational Continual Learning to better capture parameter–data dependencies. However, the effectiveness of regularization-only methods depends on whether new tasks reuse the same parameters as old ones; when sequentially introduced

attacks induce large feature-distribution shifts, importance estimates from earlier stages may no longer track which weights are useful for retaining historical decisions, motivating a systematic comparison against memory-based alternatives under the conditions encountered in NIDS deployment.

2.1.2. Memory-based approaches

Memory-based (or “episodic memory”) methods retain a subset of past training samples and rehearse them alongside the current data stream, approximating joint optimization over past and current tasks under a constrained memory budget [17,21–24]. The simplest realization, Experience Replay (ER), maintains a fixed-size buffer and mixes replayed samples with new ones in each mini-batch. Averaged Gradient Episodic Memory (A-GEM) [14] uses the buffer to compute a reference gradient that constrains the current update via a closed-form projection, rather than mixing batches at the loss level. Dark Experience Replay + + (DER + +) [15] augments label rehearsal with a logit-matching distillation term, retaining both labels and the model’s prior soft predictions on each buffered sample.

In NIDS-related studies, replay-based strategies have been reported to improve accuracy and stability over regularization alone in several incremental scenarios [25–28]; however, these studies typically evaluate a single dataset, a single base architecture, or a balanced regime, leaving the relative ranking under realistic operational conditions under-examined. To address severe class imbalance in replay buffers, class-balanced reservoir sampling (CBRS) and its extensions have been proposed [22,29], demonstrating improved retention of minority attack classes. Adjacent lines of work such as Few-Shot Class-Incremental Learning (FSCIL) [30,31] target settings where only a handful of samples of each new attack are available; while complementary, this regime is outside the scope of the present work, which assumes labeled samples of each new attack class are available at the corresponding stage.

2.2. Challenges in IoT NIDS

The CL strategies outlined above have shown promise in general contexts but applying them to NIDS introduces domain-specific constraints related to data properties and resource limitations.

2.2.1. Class imbalance and concept drift

A dataset suffers from class imbalance when the number of samples across different classes is disproportionate. This can bias a model toward the majority class, causing it to misclassify the minority class. Network traffic data is inherently imbalanced, as benign traffic typically outweighs malicious traffic by a significant margin, making it difficult for models to learn to detect attacks effectively [29,32]. Therefore, a critical step in training a NIDS is to account for this imbalance, often by applying techniques to resample or re-weight the data to create a more balanced class distribution.

Moreover, the landscape of cyber-threats is constantly evolving, a phenomenon known as concept drift. As attackers develop new methods—ranging from polymorphic malware to rapidly mutating bot-net variants [33]—the statistical properties of network traffic change, causing static, pre-trained detection models to become outdated and ineffective [3,16]. CL presents a natural solution to this problem. It allows models to be periodically updated with new data, enabling them to adapt to novel threats while retaining knowledge of past attacks [18,29] and improving their ability to generalize to unseen attack patterns [26].

2.2.2. Feature representation for IoT NIDS

Another fundamental challenge in IoT NIDS lies in designing feature representations that are both expressive and robust to the heterogeneity of IoT traffic. To this end, many practical systems rely on protocol-agnostic statistical feature extraction, which avoids dependence on specific packet formats or payload semantics. A representative example is the Kitsune feature extractor [9], employed in the UQ-IoT-IDS-2021 dataset [8], which constructs multi-scale temporal-statistical

features (e.g., packet size variance and inter-arrival time statistics) over sliding time windows. Such representations effectively capture characteristic traffic behaviors associated with attacks such as flooding or scanning, while maintaining low computational overhead and strong generalization across diverse IoT protocols.

In parallel, prior work has explored the application of deep learning models—including recurrent architectures such as LSTMs and Transformer-based encoders—to analyze sequential traffic representations for intrusion detection. Survey studies indicate that these models are typically applied to pre-extracted statistical or flow-level representations, rather than learning features directly from raw packet contents [34]. Accordingly, deep learning-based approaches are commonly used to exploit temporal structures within existing protocol-agnostic feature spaces, complementing rather than replacing handcrafted statistical representations.

2.3. Research gap

Although CL techniques have been increasingly explored for network intrusion detection [35], what remains missing is a systematic evaluation that characterizes how *existing* CL strategies behave in the IoT NIDS domain under operationally motivated conditions, including a class-imbalanced (attack-rare) sampling regime. Specifically, prior studies typically (i) evaluate strategies in isolation rather than side by side, leaving regularization-vs-replay behavior and combined methods such as EWC + ER unresolved; (ii) rely on a single dataset and base architecture under balanced sampling, with metrics often limited to F1 rather than PR-AUC, FPR at high recall, per-attack F1, and an explicit forgetting rate; (iii) implicitly adopt a class-incremental framing despite the fixed binary detection objective and within-class distribution shifts of operational NIDS, which are more naturally captured as domain-incremental learning; and (iv) seldom contrast supervised CL with unsupervised anomaly-detection baselines or examine deployment-oriented sensitivity to replay buffer size, regularization strength, and task ordering. Rather than proposing a new continual-learning algorithm, this study addresses these gaps through a unified empirical evaluation of eight existing CL strategies on three datasets and three deep learning architectures under a domain-incremental NIDS setting, with explicit balanced and unbalanced regimes, sensitivity analyses on the regularization strength, replay buffer size, and task order, and a direct comparison against three unsupervised anomaly-detection baselines (KitNET, Isolation Forest, and an autoencoder); the contribution is a systematic characterization of when and why each strategy succeeds or fails in this domain.

3. Methodology

This section details the proposed Continual Learning framework for NIDS, designed to evaluate CL strategies in IoT environments. Building on the research gap identified in Section 2, the architecture is modular, consisting of a data processing pipeline, continual learning strategies, and an incremental training protocol that rigorously evaluates knowledge retention in DIL scenarios.

3.1. Data processing pipeline

We evaluate three datasets that span IoT-specific and enterprise traffic settings: UQ-IoT-IDS-2021 [8], Kitsune [9], and CIC-IDS-2017 [10]. The same four-stage pipeline is applied to each dataset (per-dataset configurations are detailed in Section 4.1):

- **Feature Extraction:** Each dataset’s raw traffic is converted into a fixed-dimensional flow- or packet-level statistical feature vector using the extractor associated with the dataset’s release format. No raw payload bytes or high-cardinality categorical metadata are propagated into the feature space.
- **Preprocessing:** Features are cast to numeric representations and standardized with a `StandardScaler` whose mean and variance are

estimated *once* on the Stage 1 training set and reused unchanged for all subsequent stages, including Stage t training and test data, and replay-buffer samples. Rows with non-finite (e.g., NaN or Inf) values are removed, and any columns that potentially bias the model toward shortcut learning (e.g., timestamp, IP address) are dropped.

- **Label Mapping:** The detection task is formulated as binary classification, with all attack variants mapped to a positive class ($y = 1$) and benign traffic to a negative class ($y = 0$). This formulation keeps the label space *fixed* across stages, while newly emerging attack types manifest as distributional shifts within the attack class, aligning the problem with a DIL setting.
- **Imbalance Handling:** To handle the heavy class imbalance characteristic of network traffic, we evaluate two training-data regimes for every dataset. In the *balanced* regime, each class with raw count above a per-dataset floor is downsampled to that floor while rarer classes are retained in full; this regime controls for class-imbalance effects and acts as a stress test for the plasticity–stability trade-off, allowing any performance degradation to be attributed to forgetting rather than data scarcity. In the *unbalanced* regime, we instead apply tier-based caps under which the Normal class dominates the dataset—reflecting the attack-rare nature of real network traffic—while major attack classes are capped at a smaller share and rare attack classes are kept in full, approximating operational skew. The same downsampling logic and CL pipeline are applied uniformly across the three datasets; per-class floors, caps, and counts under both regimes are reported in Section 4.1.

3.2. Continual learning framework

We start with *Weight Inheritance* (WI) as a lower-bound baseline for CL, which fine-tunes the model from the previous stage's weights without any explicit forgetting-mitigation mechanism. We then evaluate six CL strategies that span the three CL families identified in the literature [6]: regularization-based methods (*Elastic Weight Consolidation*, EWC; *Synaptic Intelligence*, SI; *Learning without Forgetting*, LwF), memory-based methods (*Experience Replay*, ER; *Averaged Gradient Episodic Memory*, A-GEM), and a hybrid method (*Dark Experience Replay* + +, DER + +). We additionally include *joint training* as a non-CL upper bound that retrains on the cumulative class set at every stage and therefore does not suffer from forgetting.

Three neural architectures are considered as base models for CL—a Multi-Layer Perceptron (MLP), a bidirectional LSTM, and a lightweight Transformer encoder—so that the effects of each CL mechanism can be observed across feed-forward and sequence-modelling capacities. Random Forest (RF) and Support Vector Machine (SVM) classifiers are retained as conventional non-continual baselines for comparison.

Throughout this subsection, we let $t = 1, \dots, T$ index the sequential training stages, D_t denote the data stream observed at stage t , and $\theta \in \mathbb{R}^P$ denote the (flattened) trainable parameter vector of the model, with scalar entries θ_i for $i = 1, \dots, P$. The cross-entropy loss is written as $\mathcal{L}_{CE}$.

3.2.1. Weight inheritance (WI, naive fine-tuning baseline)

In this strategy, the model's structure and learned weights are carried forward from one training stage to the next. The model then resumes training using only the data from the new, current stage. No additional mechanisms are used to explicitly retain previous knowledge. WI is therefore equivalent to the standard *naive fine-tuning* baseline used in the continual learning literature: a warm-started network trained on each new task without any forgetting-mitigation mechanism. In our evaluation, WI serves as the lower-bound CL reference against which all forgetting-mitigation strategies (EWC, SI, LwF, ER, DER + +, A-GEM) are measured.

As illustrated in Fig. 1, the model at Stage 2 ($M^{(2)}$) is initialized with the weights (W_1) from Stage 1. These weights are then updated using the

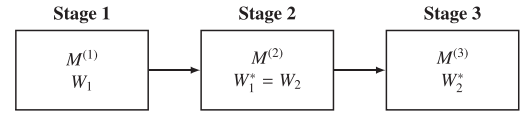


Fig. 1. Weight Inheritance Strategy.

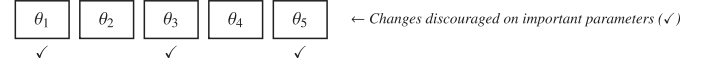


Fig. 2. Elastic Weight Consolidation Strategy.

new task data, resulting in a new set of weights (W_1^*). This process repeats for each subsequent stage, making this method a useful benchmark to measure the severity of catastrophic forgetting.

WI is used as the default initialization strategy across incremental stages. Other strategies (e.g., EWC and ER) are applied on top of WI to introduce regularization- and replay-based mechanisms.

3.2.2. Elastic weight consolidation (EWC)

EWC [4] is a regularization-based approach that addresses forgetting at the parameter level. It works by constraining updates to the model's weights that are deemed most important for performance on previous tasks. After training at each stage, EWC estimates the importance of each parameter via the Fisher Information Matrix, computed by sampling targets from the model's predictive distribution and averaging the squared per-parameter gradients of the resulting log-likelihood.

In the subsequent stage, the loss function is updated with a loss term that discourages significant changes to important weights, while allowing less critical weights to be updated more freely to learn new patterns (see Fig. 2).

The total loss function at stage $t + 1$ is thus defined as:

$$\mathcal{L}_{EWC} = \mathcal{L}_{CE}(D_{t+1}) + \lambda \sum_i \bar{F}_{ii}^{(t)} (\theta_i - \theta_i^{*(t)})^2,$$

where $\mathcal{L}_{CE}(D_{t+1})$ is the standard cross-entropy loss on the current stage's data stream, the index i runs over the scalar parameters of the model, and the regularization term penalizes large changes to parameters that were important for previously seen tasks. To support more than two stages, $\bar{F}^{(t)}$ is maintained as a running average of the per-stage Fisher matrices $F^{(t)}$ (each computed at the end of its respective stage) and is re-anchored to the consolidated parameter values $\theta^{*(t)}$ at the end of every stage:

$$\bar{F}^{(t)} = \frac{t-1}{t} \bar{F}^{(t-1)} + \frac{1}{t} F^{(t)}.$$

Here $\bar{F}_{ii}^{(t)}$ is the diagonal entry of $\bar{F}^{(t)}$ corresponding to parameter θ_i , and the hyperparameter λ controls the strength of the penalty, balancing the preservation of old knowledge against the acquisition of new information. Conceptually, each summand in the regularizer can be read as a product of three factors: a weight (λ), the parameter's importance ($\bar{F}_{ii}^{(t)}$), and the squared change in that parameter from its consolidated anchor ($(\theta_i - \theta_i^{*(t)})^2$).

3.2.3. Synaptic intelligence (SI)

SI [11] is a regularization-based alternative to EWC that estimates parameter importance *online* during training, rather than via a separate Fisher Information pass at the end of each stage (see Fig. 3). $\theta_i^{(t)}$ is the value of θ_i at the end of stage t .

For every parameter θ_i , SI accumulates a path integral of the per-step loss reduction along the optimization trajectory within stage t :

$$\omega_i^{(t)} = \sum_{\tau \in S_t} -g_{i,\tau} \Delta \theta_{i,\tau},$$

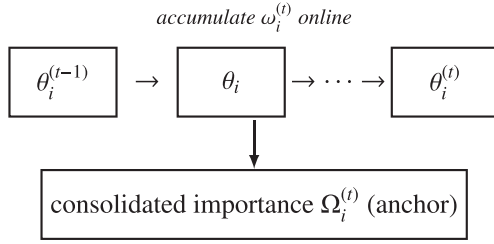


Fig. 3. Synaptic Intelligence Strategy.

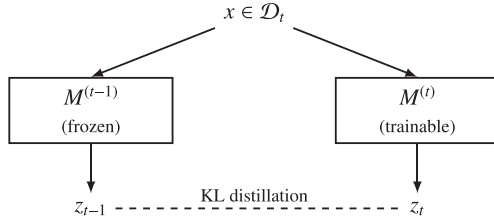


Fig. 4. Learning without Forgetting Strategy.

where S_t denotes the set of optimization steps taken during stage t , and $g_{i,\tau}$ and $\Delta\theta_{i,\tau}$ are, respectively, the unregularized task gradient and the parameter update at step τ . After stage t , the path integral is normalized by the squared per-stage displacement $\Delta_i^{(t)} = \theta_i^{(t)} - \theta_i^{(t-1)}$ and added to the running consolidated importance:

$$\Omega_i^{(t)} = \Omega_i^{(t-1)} + \frac{\omega_i^{(t)}}{(\Delta_i^{(t)})^2 + \xi},$$

where ξ is a small damping constant that prevents division-by-zero when a parameter does not move. The consolidated importance $\Omega_i^{(t)}$ is then used as a quadratic anchor analogous to EWC's Fisher term:

$$\mathcal{L}_{SI} = \mathcal{L}_{CE}(\mathcal{D}_{t+1}) + \lambda_{SI} \sum_i \Omega_i^{(t)} (\theta_i - \theta_i^{*(t)})^2,$$

where $\theta_i^{*(t)}$ is the anchored value of parameter θ_i at the end of stage t , and λ_{SI} is a hyperparameter that controls the strength of the SI penalty. Compared with EWC, SI avoids the additional gradient pass required for Fisher estimation but stores per-parameter importance, anchor, and trajectory tensors, leading to a slightly larger memory footprint.

3.2.4. Learning without forgetting (LwF)

LwF [12] is a distillation-based regularizer that does not require a replay buffer. At the end of each stage, a frozen snapshot $M^{(t-1)}$ of the previous model is retained (see Fig. 4).

While training the current model $M^{(t)}$ on $\mathcal{D}_t$, the loss is augmented with a knowledge-distillation (KD) term that encourages the soft predictions of $M^{(t)}$ on the *current-stage* inputs to match those of the snapshot:

$$\mathcal{L}_{LwF} = \mathcal{L}_{CE}(\mathcal{D}_t) + \lambda_{LwF} \cdot T^2 \text{KL}\left(\sigma\left(\frac{z_{t-1}}{T}\right) \parallel \sigma\left(\frac{z_t}{T}\right)\right),$$

where z_{t-1} and z_t denote the logits of the snapshot and the current model on the same inputs, $\sigma(\cdot)$ is the softmax operator, $\text{KL}(\cdot)$ is the Kullback–Leibler divergence between two distributions, T is a temperature hyperparameter that softens the target distribution, and λ_{LwF} controls the strength of the distillation term. Whereas the original LwF formulation restricts the distillation term to the old-task output heads in a multi-head architecture, our domain-incremental setting uses a single shared binary classifier whose output space ($\{\text{Normal}, \text{Attack}\}$) is fixed across stages. There is therefore no growing or task-specific output

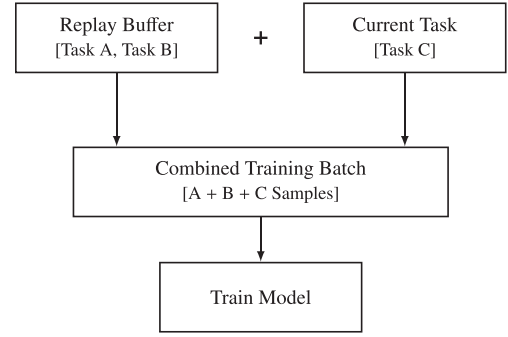


Fig. 5. Experience Replay Strategy.

partition over which the KD term must be sliced; the distillation is naturally applied over the full softmax output. Intuitively, LwF preserves the input–output mapping learned for past attack distributions through the model's response on new traffic, without storing any past samples.

3.2.5. Experience replay (ER)

ER [13] mitigates catastrophic forgetting by maintaining a fixed-size buffer that stores a small subset of samples from previously observed stages. During training on newly arriving data, each mini-batch is constructed by combining samples from the current stage with samples drawn from this replay buffer (see Fig. 5). By repeatedly re-exposing the model to historical data, ER reinforces previously learned patterns and reduces interference caused by updates from new attack instances.

In our setting, the replay buffer maintains a class-balanced subset of past *raw input samples* (rather than embeddings or synthetic representations), with a fixed per-class budget K and total storage bounded by $K \times |\mathcal{C}_{1:t}|$, where $\mathcal{C}_{1:t}$ is the set of classes observed up to stage t . At each training stage, newly observed samples are incorporated into the buffer while preserving the per-class quota, ensuring that earlier attack behaviors remain represented throughout training. This design is particularly important for intrusion detection, where malicious traffic is sparse and early-stage attack patterns may otherwise be overwritten as new attack variants emerge.

At stage t , the model is trained using the cross-entropy loss on samples from the current data stream $\mathcal{D}_t$ and on replay samples drawn from the buffer $\mathcal{B}_t$. Our implementation forwards the two batches separately and combines their cross-entropy losses with equal weight (0.5 each):

$$\mathcal{L}_{ER} = 0.5 \mathcal{L}_{CE}(\mathcal{D}_t) + 0.5 \mathcal{L}_{CE}(\mathcal{B}_t),$$

where $\mathcal{L}_{CE}(\cdot)$ denotes the cross-entropy loss evaluated on the indicated batch. Through this mechanism, ER approximates joint training over past and current data under a constrained memory budget.

3.2.6. Averaged gradient episodic memory (A-GEM)

A-GEM [14] is a memory-based method that uses the replay buffer for gradient projection rather than direct loss mixing. At each optimization step, A-GEM computes both the current-task gradient g and a reference gradient g_{ref} on a single mini-batch randomly sampled from the replay buffer. Unlike its predecessor GEM, which enforces one inequality constraint per past task at every step, A-GEM averages over this single buffer batch, reducing the per-step optimization to a closed-form projection (see Fig. 6).

If the two gradients are misaligned (i.e., $\langle g, g_{\text{ref}} \rangle < 0$), g is projected onto the half-space that does not increase the replay-buffer loss; otherwise, g is used unchanged:

$$\tilde{g} = \begin{cases} g - \frac{\langle g, g_{\text{ref}} \rangle}{\langle g_{\text{ref}}, g_{\text{ref}} \rangle} g_{\text{ref}} & \text{if } \langle g, g_{\text{ref}} \rangle < 0, \\ g & \text{otherwise.} \end{cases}$$

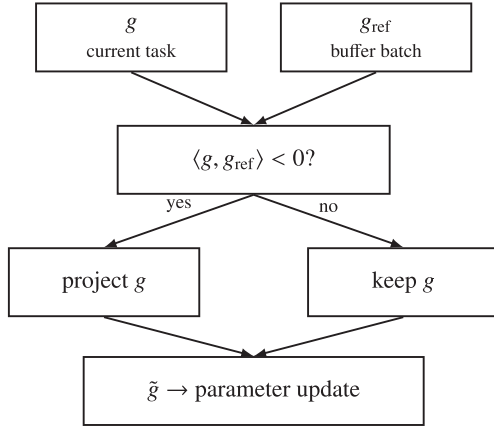


Fig. 6. Averaged Gradient Episodic Memory Strategy.

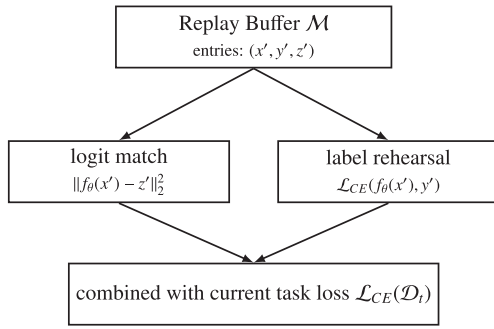


Fig. 7. Dark Experience Replay ++ Strategy.

The (possibly projected) gradient $\tilde{g}$ is then used for the parameter update. A-GEM imposes only a constraint on the update direction and does not require explicit reweighting of past versus current data, while sharing the same buffer storage as ER.

3.2.7. Dark experience replay ++ (DER++)

DER++ [15] is a hybrid memory-based method that augments standard ER with a distillation term. The replay buffer $\mathcal{B}$ stores entries of the form (x', y', z') , where (x', y') is a past sample with its label and z' is a logit vector used as a soft distillation target in subsequent stages; the prime notation distinguishes buffer entries from current-stage samples (x, y) . In our implementation, z' is refreshed at the end of every stage by re-evaluating the just-trained model on all current buffer entries, so the distillation target at stage $t+1$ corresponds to the model's response at the end of stage t (see Fig. 7).

During training at stage t , replay samples contribute two additional terms to the loss: a soft logit-matching term and a hard label-classification term:

$$\mathcal{L}_{DER++} = \mathcal{L}_{CE}(D_t) + \alpha \mathbb{E}_{(x', y', z') \sim \mathcal{M}} [\|f_\theta(x') - z'\|_2^2] + \beta \mathbb{E}_{(x', y', z') \sim \mathcal{M}} [\mathcal{L}_{CE}(f_\theta(x'), y')],$$

where $f_\theta(\cdot)$ is the current model, $\mathcal{M}$ is the replay buffer, and α, β control the relative weights of logit distillation and label rehearsal. Storing logits captures the model's most recently learned decision surface in addition to ground-truth labels, which has been shown to improve stability under non-stationary distributions.

3.2.8. Joint training (Upper bound)

Joint training is included as a non-CL reference. At every stage t , a fresh model is trained from scratch on the cumulative dataset $D_{1:t}$ containing all classes observed up to and including stage t (see Fig. 8).

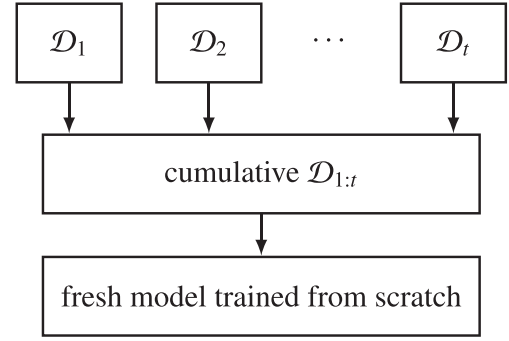


Fig. 8. Joint Training Strategy.

Because the joint training sees all past data simultaneously, it does not suffer from catastrophic forgetting; its performance therefore serves as an empirical ceiling against which the gap incurred by each CL method can be quantified. Joint training is computationally and memory-prohibitive in operational IoT NIDS settings, where retaining and re-training on the full traffic history at every update is exactly the burden CL methods are designed to avoid.

3.3. Incremental training protocol

Continual learning is conventionally categorized into three settings [5,6]: *class-incremental learning* (CIL), in which the label space $\mathcal{Y}_t$ grows as new classes are revealed at each stage and the model must distinguish all classes seen so far; *task-incremental learning* (TIL), in which the label space is partitioned across tasks and a task identifier is supplied at inference; and *domain-incremental learning* (DIL), in which the label space $\mathcal{Y}$ is fixed and shared across stages while the input marginal $P_t(X)$ shifts over time. We adopt the DIL formulation, motivated by the operational character of NIDS deployment: alert or no alert.

We frame intrusion detection as binary classification with the fixed label space $\mathcal{Y} = \{\text{Normal}, \text{Attack}\}$, consistent with how production sensors typically operate—issuing an alert versus no alert—and treat attack-family attribution as a separate downstream forensics task. Under this framing, introducing a new attack type at stage t does not extend $\mathcal{Y}$; instead, it shifts the within-class input marginal $P_t(X | Y = \text{Attack})$, because each new attack technique produces a distinct point cloud in the feature space. The Normal distribution $P_t(X | Y = \text{Normal})$ is assumed to be approximately stable across stages. This is the canonical DIL signature, and it is what our protocol measures. A multi-class formulation in which specific attack types are predicted directly is feasible but is left to attribution-oriented work, as our scope is detection rather than identification.

Attack-family labels play two roles in this work, and neither requires them during inference. First, they serve as an experimental grouping key: the stage schedule (Section 4.4) partitions each dataset by attack family to simulate the sequential emergence of new attacks. Second, they act as the sampling key for the class-balanced replay buffer, which is the only point where the method utilizes them in operation; here we assume security analysts can attribute a newly identified attack to its family within a reasonable period. The deployed detector uses only the binary Normal/Attack labels and never predicts an attack-family label, keeping the task binary DIL.

We instantiate the DIL stream via a sliding-window training protocol. At each stage t , the training set D_t contains the invariant Normal class together with the two most recently introduced attack types $\{A_{t-1}, A_t\}$, with $A_0 = \emptyset$ at the initial stage so that D_1 contains only Normal and A_1 . Past-stage attack samples are not reused except via the buffer $\mathcal{M}$ maintained by replay-based CL strategies. After training at stage t , the model is evaluated on the cumulative test set $D_{1:t}^{\text{test}}$, which contains held-out

Table 1
Class Distribution: Raw, Balanced, and Tier-Based unbalanced Sampling.

Class	Raw	Balanced	Unbalanced
Normal	17,422,159	15,302	400,000
ACK Flooding	8,910,000	15,302	50,000
UDP Flooding	8,561,564	15,302	50,000
SYN Flooding	5,521,169	15,302	50,000
HTTP Flooding	683,408	15,302	50,000
Telnet-brute Force	128,653	15,302	50,000
Service Detection	92,644	15,302	50,000
Port Scanning	41,624	15,302	41,624
Host Discovery	25,775	15,302	25,775
ARP Spoofing	15,302	15,302	15,302
TOTAL	41,402,298	153,020	782,701

samples for every attack type observed up to stage t . The deliberate mismatch between the local (sliding) training distribution and the global (cumulative) evaluation distribution is what exposes catastrophic forgetting. The per-stage class lists for the three datasets evaluated in this work follow a Cyber Kill Chain-inspired ordering and are listed explicitly in Section 4.4.

4. Experimental setup

4.1. Datasets configuration

4.1.1. UQ-IoT-IDS-2021

We utilize the UQ-IoT-IDS-2021 dataset [8], which captures network traffic from real IoT devices under various attack scenarios. Raw packet captures are converted into 100 temporal-statistical features using the Kitsune feature extractor [9], which summarizes flow-level behaviors (e.g., packet size statistics and inter-arrival times) across five exponentially weighted time windows (100 ms, 500 ms, 1.5 s, 10 s, and 1 min); the current packet length is appended to yield the per-packet feature vector. As shown in Table 1, this dataset exhibits extreme class imbalance; for example, the Normal class contains over 17 million samples whereas ARP Spoofing includes only 15,302 samples.

We construct a *balanced* training set by downsampling each class with a raw count above a 15,302-sample floor to exactly 15,302 samples, while all available samples are retained for the ARP Spoofing class. We additionally construct a *tier-based unbalanced* training set in which the Normal class is capped at 400,000 samples, major attack classes (raw count > 50,000) are capped at 50,000 samples, and the remaining minor classes (Port Scanning, Host Discovery, ARP Spoofing) are kept in full.

4.1.2. Kitsune

We additionally evaluate on the Kitsune dataset [9], which was captured from either an IoT network testbed or a surveillance system. It provides nine separate captures, each containing roughly one million clean (pre-attack) packets followed by an attack episode of one of nine attack types: ARP MitM, Active Wiretap, Fuzzing, Mirai Botnet, OS Scan, SSDP Flood, SSL Renegotiation, SYN DoS, and Video Injection. Each packet is represented by a 115-dimensional feature vector extracted by Kitsune's incremental damped-incremental statistics extractor (per-channel statistics over five time windows). Per-attack capture files contain both benign and attack packets; we therefore restrict the *Normal* class to the leading clean window (the first 10^6 packets) of each capture and discard benign-labeled packets that appear after the attack starts, as those may include collateral, attack-adjacent traffic. Benign rows from all nine captures are then pooled into a single *Normal* class, while attack rows from each capture form one labeled class. As shown in Table 2, the resulting raw distribution is dominated by the Normal class (8,121,620 samples), and the rarest attack class (SYN DoS, 7038 samples) spans nearly four orders of magnitude below it. Using the same protocol, we construct a *balanced* training set by downsampling each eligible class to

Table 2

Kitsune class Distribution: Raw, Balanced, and Tier-Based unbalanced Sampling. Raw *Normal* count reflects only the pre-attack clean window (first 10^6 packets per capture, pooled across the nine captures); benign-labeled packets occurring after the attack starts in each capture are discarded.

Class	Raw	Balanced	Unbalanced
Normal	8,121,620	10,000	400,000
SSDP Flood	1,439,604	10,000	50,000
ARP MitM	1,145,272	10,000	50,000
Active Wiretap	923,216	10,000	50,000
Mirai Botnet	642,516	10,000	50,000
Fuzzing	432,783	10,000	50,000
Video Injection	102,499	10,000	50,000
SSL Renegotiation	92,652	10,000	50,000
OS Scan	65,700	10,000	50,000
SYN DoS	7038	7038	7038
TOTAL	12,972,900	97,038	807,038

a 10,000-sample floor, retaining SYN DoS in full as a rare class, and a *tier-based unbalanced* training set in which the Normal class is capped at 400,000 samples, major attack classes (raw count > 50,000) are capped at 50,000 samples, and the remaining minor class (SYN DoS) is kept in full.

4.1.3. CIC-IDS-2017

To assess the generalizability of our findings beyond IoT-specific traffic, we additionally evaluate on the CIC-IDS-2017 dataset [10], an enterprise-traffic intrusion detection benchmark released by the Canadian Institute for Cybersecurity. The dataset captures five days of synthetic network activity covering fourteen attack categories, each labeled with bidirectional flow-level statistics extracted by CICFlowMeter [36] (e.g., flow duration, inter-arrival times, packet length statistics, TCP flag counts, subflow byte/packet counts, active/idle time intervals). To prevent the model from learning spurious shortcuts, we exclude the Timestamp column—which would induce temporal bias from the day-by-day capture protocol—and the Destination Port column, which would let the model memorize port-specific attack signatures (e.g., FTP-Patator → 21, SSH-Patator → 22, Heartbleed → 443) rather than learn generalizable flow behavior. The remaining 67 numerical features are retained. Rows containing NaN or infinite values—most commonly produced by zero-duration flows in the Flow Bytes/s and Flow Packets/s columns—are removed. To match the nine-stage Cyber Kill Chain progression (Table 6), the three *Web Attack* variants (Brute Force, XSS, SQL Injection) are merged into a single *Web Attack* class, and the four DoS variants (slowloris, Slowhttptest, GoldenEye, Hulk) are merged into a single *DoS* class, yielding ten classes (one Benign and nine attacks).

CIC-IDS-2017 exhibits even more extreme class imbalance than UQ-IoT-IDS-2021 and Kitsune, from over two million Benign flows down to only 11 Heartbleed flows. We construct a *balanced* training set by downsampling each class with a raw count above a 10,000-sample floor to exactly 10,000 samples, while classes below the floor (FTP-Patator, SSH-Patator, Web Attack, Bot, Infiltration, Heartbleed) are retained in full. We additionally construct a *tier-based unbalanced* training set in which the Benign class is capped at 50,000 samples, major attack classes (raw count > 15,000) are capped at 15,000 samples, and remaining minor classes are kept in full (Table 3).

4.2. Implementation details

All experiments were implemented using PyTorch 2.4.1 [37] and Scikit-learn [38] on a workstation equipped with an NVIDIA GeForce RTX 4080 SUPER GPU and an Intel Core i9-14900K CPU, running Ubuntu 24.04 LTS.

Table 3

CIC-IDS-2017 class Distribution: Raw, Balanced, and Tier-Based unbalanced Sampling. Raw counts are reported after merging the three Web attack variants into *Web_Attack* and the four DoS variants into *DoS*.

Class	Raw	Balanced	Unbalanced
Benign	2,071,822	10,000	50,000
DoS	193,623	10,000	15,000
DDoS	128,014	10,000	15,000
PortScan	90,694	10,000	15,000
FTP-Patator	5931	5931	5931
SSH-Patator	3219	3219	3219
Web Attack	2143	2143	2143
Bot	1948	1948	1948
Infiltration	36	36	36
Heartbleed	11	11	11
TOTAL	2,497,441	53,288	108,288

4.3. Model configurations

We evaluate three neural architectures (MLP, LSTM, and Transformer) as base models for the continual learning strategies, complemented by two non-continual classical baselines (RF, SVM) and three unsupervised anomaly detectors for reference. For the MLP, RF, and SVM, the architecture and key hyperparameters were obtained via a grid search on the balanced UQ-IoT-IDS-2021 training set, with the configuration achieving the highest mean F1-score retained. For

the LSTM and Transformer baselines, we adopt commonly used architectural defaults from the deep-learning literature (bidirectional layers, multi-head attention, pre-LayerNorm) and align their hidden dimension, optimizer, batch size, and training horizon with the tuned MLP recipe so that performance differences reflect each architecture's inductive bias rather than its training budget. The final hyperparameter values used in all main experiments are summarized in Table 4; the same configurations are reused unchanged across the UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017 evaluations to isolate the effect of dataset distribution from per-dataset tuning. Within each stage, 10% of the training partition is held out as a stratified validation set on which we monitor F1 after every epoch; the best-F1 checkpoint is retained for evaluation, and training stops early if validation F1 fails to improve for three consecutive epochs (rarely binding at the 5-epoch cap, but applied uniformly across all CL strategies).

Our pipeline contains several sources of randomness: weight initialization, mini-batch shuffling, replay-buffer subsampling, and the train/test split. We report the headline results (Table 7) with mean $\pm$ standard deviation over five random seeds ($\{0, \dots, 4\}$), where a single seed drives all sources jointly. All randomness is explicitly seeded.

We additionally include three unsupervised anomaly detectors as references—KitNET [9], an Isolation Forest, and an autoencoder—each trained only on Normal traffic and flagging attacks by thresholding an anomaly score. Each is trained once at Stage 1 on the same Normal pool the continual learners update on and is reused unchanged.

Table 4

Final hyperparameters used in the main continual learning experiments. The same configurations are applied across UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017 unless noted otherwise.

Model	Hyperparameter	Value
MLP	Hidden layers $\times$ neurons	3×256
	Activation	ReLU
	Optimizer	Adam, $\text{lr} = 10^{-3}$
	Epochs / batch size	5 / 64
	Output dimension	2 (binary)
LSTM	Layers / hidden size	2 / 256
	Direction	Bidirectional
	Recurrent dropout	0.2
	Tokenization	one feature value per timestep
	Optimizer	Adam, $\text{lr} = 10^{-3}$
	Epochs / batch size	5 / 64
Transformer	Encoder layers	1
	d_{model} / heads / FFN dim	256 / 4 / 256
	Normalization	Pre-LN
	Position embedding	learned, per-feature
	Tokenization	one feature value per timestep
	Optimizer	Adam, $\text{lr} = 10^{-3}$
	Epochs / batch size	5 / 64
RF	Trees / max depth	50 / 40
	Min. samples to split	2
	Training schedule	retrained from scratch each stage
SVM	Kernel approximation	Nyström, 1000 components, $\gamma = 0.1$
	Classifier	Linear, $C = 1000$
	Training schedule	retrained from scratch each stage
Anomaly detectors (shared settings)	Detection threshold	F1-optimal point on the labeled validation slice
	Training schedule	once at Stage 1, fixed afterwards
KitNET	Max autoencoder size / lr	10 / 0.1 (UQ-IoT-IDS-2021); 5 / 0.1 (Kitsune); 10 / 0.05 (CIC-IDS-2017)
	Grace periods (FM / AD)	25% / 75% of the fit samples
Isolation Forest	Trees	200 (all datasets)
Autoencoder	Bottleneck width	16 (UQ-IoT-IDS-2021, CIC-IDS-2017); 32 (Kitsune)
	Architecture	input $\rightarrow$ 64 $\rightarrow$ bottleneck $\rightarrow$ 64 $\rightarrow$ input, ReLU
	Optimizer	Adam, $\text{lr} = 10^{-3}$
	Epochs / batch size	20 / 256

4.3.1. Base models for CL

- **MLP:** The MLP search space ranged over $\{1, 2, 3\}$ hidden layers and hidden dimensions in $\{32, 64, 128, 256, 512, 1024\}$ neurons. The selected configuration consists of an input layer matching the dataset's feature dimensionality (UQ-IoT-IDS-2021: 100, Kitsune: 100, CIC-IDS-2017: 67), three hidden layers with 256 ReLU-activated neurons each, and an output layer with two neurons for binary (Normal vs. Attack) classification. Adam [39] is used for training with a learning rate of 10^{-3} . At each incremental stage, the model is trained for five epochs with a batch size of 64.
- **LSTM:** A bidirectional, 2-layer LSTM with hidden size 256 and recurrent dropout 0.2. Each input feature vector is treated as a length- d sequence of scalar tokens (one feature value per timestep), and the final forward and backward hidden states from the top layer are concatenated and passed to a linear classifier. The bidirectional, multi-layer design corrects a directional bias observed in early pilot runs, where a forward-only single-layer variant systematically over-weighted later features in the (semantically arbitrary) input ordering. The optimizer, learning rate, batch size, and training horizon match the MLP.
- **Transformer:** A single Transformer encoder layer with model dimension 256, four attention heads, feed-forward dimension 256, and pre-LayerNorm (norm_first = True). Inputs use the same per-feature tokenization as the LSTM; a learned position embedding is added before encoding, and the per-token outputs are mean-pooled before the final linear classifier. Pre-LN is used because the post-LN variant collapsed to a constant predictor in late CL stages without learning-rate warmup. The optimizer, learning rate, batch size, and training horizon match the MLP.

4.3.2. Non-continual baselines

- **RF:** We tuned the number of decision trees $\{50, 100, 200, 400\}$, the maximum depth $\{10, 20, 30, 40\}$, and the minimum samples required to split a node $\{2, 5, 10, 20\}$. The final model employs an ensemble of 50 decision trees with a maximum depth of 40 and the minimum samples required to split a node of 2, retrained from scratch at each stage.
- **SVM:** The tuning process evaluated linear SVM and kernel approximation methods (RBF Sampler and Nyström). The search space included parameters $C \in \{0.01, 0.1, 1, 10, 100, 1000\}$, $\gamma \in \{0.01, 0.1, 1\}$, and approximation components in $\{500, 1000\}$. The chosen configuration uses the Nyström method with 1000 components and $\gamma = 0.1$, followed by a linear classifier with $C = 1000$, retrained from scratch at each stage.
- **Anomaly detectors (KitNET, Isolation Forest, Autoencoder):** A stratified 10% slice of the training partition, containing Normal and all attack families, is held out as a labeled validation set; the detector is fit on the remaining Normal traffic, and its decision threshold is set to the value that maximizes F1 on the held-out slice, chosen jointly with each detector's main hyperparameter (KitNET: max autoencoder size $\{5, 10\}$, learning rate $\{0.05, 0.1\}$; Isolation Forest: trees $\{100, 200\}$; autoencoder: bottleneck $\{16, 32\}$). Rather than a single fixed cutoff, the selected thresholds span a wide range of validation-normal-score percentiles across detectors and datasets (from below the 1st to about the 91st), reflecting how differently each detector separates attacks from Normal. Anomaly scores are the ensemble reconstruction error (KitNET), per-sample reconstruction MSE (autoencoder), and negative isolation score (Isolation Forest).

4.3.3. CL strategies

Each base architecture (MLP, LSTM, Transformer) is paired with the six CL strategies introduced in Section 3.2—EWC, SI, LwF, ER, A-GEM, and the hybrid DER++—together with the WI fine-tuning baseline and the joint-training upper bound. WI is the warm-started naive fine-tuning

Table 5

Default hyperparameters for the continual learning strategies. All strategies are applied on top of WI and share the base-model training recipe in Table 4.

Strategy	Hyperparameter	Value
EWC	Regularization coefficient λ_{EWC}	0.4
SI	Regularization coefficient λ_{SI}	1.0
	Damping ξ	0.1
LwF	Distillation weight λ_{LwF}	1.0
	Temperature T	2.0
ER	Replay mini-batch size	32
	Buffer budget K per class	100
A-GEM	Reference mini-batch size	32
	Buffer budget K per class	100 (shared with ER)
DER++	Distillation weight α	0.5
	CE replay weight β	0.5
	Buffer budget K per class	100 (shared with ER)

baseline and introduces no additional hyperparameters; all other strategies are applied on top of WI and share the base-model training recipe in Table 4. Unless otherwise specified in the sensitivity analysis, the default hyperparameter values for each strategy are summarized in Table 5.

All strategies follow the formulations in Section 3.2; the only implementation choice not fully specified there is the construction of the replay buffer shared by ER, A-GEM, and DER++. ER, A-GEM, and DER++ share a single per-class replay buffer of raw input samples. At the end of each training stage t , for every attack class observed in the current sliding window, we draw up to K samples uniformly at random without replacement from that class's stage- t training partition; classes with fewer than K samples are kept in full. Each class occupies its own slot in the buffer keyed by class label, so a class re-appearing in a later window overwrites its previous slot (effectively a class-balanced reservoir with one slot per class), while classes that have rotated out of the window retain their most recent subsample. Total replay storage is therefore bounded by $K \times |C_{1:t}|$.

4.4. Sequential task definition

To simulate the domain-incremental scenario, we define a sequence of stages in which new attack types are introduced over time on each evaluated dataset. Rather than introducing attacks in an arbitrary order, the task sequence on every dataset is designed to reflect an attack lifecycle-inspired progression, ranging from reconnaissance and access-related attacks to traffic-based disruption and protocol-level manipulation.

At every stage t , the training set $\mathcal{D}_t$ is constructed using the invariant “Normal” class and the two most recent attack types (A_t, A_{t-1}); at the initial stage $t = 1$, only the newly introduced attack type (A_1) is available, since no prior attack has been observed. This “sliding window” protocol ensures the model is constantly adapting to the latest threats. The cumulative test set $\mathcal{D}_{1:t}^{\text{test}}$ evaluates the model's ability to retain knowledge of all previously encountered attacks ($C_{1:t}$). Each fold is split into training (80%) and test (20%) subsets; only training samples are used to construct $\mathcal{D}_t$, while test samples are added cumulatively to $\mathcal{D}_{1:t}^{\text{test}}$ to prevent data leakage. The full nine-stage progression for UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017 are listed in Table 6; in all three cases the test set at stage t is $C_{1:t} = C_{1:t-1} \cup \{A_t\}$ with $C_{1:1} = \{\text{Normal}, A_1\}$.

The progressions follow the Cyber Kill Chain logic: on UQ-IoT-IDS-2021, reconnaissance (Stages 1–3), access and lateral movement (Stages 4–5), exploitation or botnet recruitment (Stages 6–7), and availability disruption or escalation (Stages 8–9). On Kitsune, the chain begins with OS-fingerprinting and routing manipulation (OS Scan, ARP MitM), continues with traffic interception and protocol abuse (Active Wiretap, Fuzzing, SSL Renegotiation), and ends with botnet enrolment (Mirai Botnet) followed by amplification, connection exhaustion, and content

Table 6

Sequential task Definition across the three datasets: New attack and sliding training window per stage. The cumulative test set at stage t is $C_{1:t} = C_{1:t-1} \cup \{A_t\}$ with $C_{1:1} = \{\text{Normal}, A_1\}$ (Benign on CIC-IDS-2017).

Stage (t)	New Attack (A_t)	Training Classes (C_t)
<i>UQ-IoT-IDS-2021</i>		
1	Host Discovery (Host)	{Normal, Host}
2	Port Scanning (Port)	{Normal, Host, Port}
3	Service Detection (Service)	{Normal, Port, Service}
4	ARP Spoofing (ARP)	{Normal, Service, ARP}
5	Telnet-brute Force (Telnet)	{Normal, ARP, Telnet}
6	UDP Flooding (UDP)	{Normal, Telnet, UDP}
7	SYN Flooding (SYN)	{Normal, UDP, SYN}
8	ACK Flooding (ACK)	{Normal, SYN, ACK}
9	HTTP Flooding (HTTP)	{Normal, ACK, HTTP}
<i>Kitsune</i>		
1	OS Scan	{Normal, OS_Scan}
2	ARP MitM	{Normal, OS_Scan, ARP_MitM}
3	Active Wiretap	{Normal, ARP_MitM, Active_Wiretap}
4	Fuzzing	{Normal, Active_Wiretap, Fuzzing}
5	SSL Renegotiation	{Normal, Fuzzing, SSL_Renegotiation}
6	Mirai Botnet	{Normal, SSL_Renegotiation, Mirai_Botnet}
7	SSDP Flood	{Normal, Mirai_Botnet, SSDP_Flood}
8	SYN DoS	{Normal, SSDP_Flood, SYN_DoS}
9	Video Injection	{Normal, SYN_DoS, Video_Injection}
<i>CIC-IDS-2017</i>		
1	PortScan	{Benign, PortScan}
2	FTP-Patator	{Benign, PortScan, FTP-Patator}
3	SSH-Patator	{Benign, FTP-Patator, SSH-Patator}
4	Heartbleed	{Benign, SSH-Patator, Heartbleed}
5	Web Attack	{Benign, Heartbleed, Web_Attack}
6	Infiltration	{Benign, Web_Attack, Infiltration}
7	Bot	{Benign, Infiltration, Bot}
8	DoS	{Benign, Bot, DoS}
9	DDoS	{Benign, DoS, DDoS}

injection (SSDP Flood, SYN DoS, Video Injection). On CIC-IDS-2017, it begins with port reconnaissance (PortScan), advances through credential brute-forcing on exposed services (FTP-Patator, SSH-Patator), exploits a service vulnerability for credential leakage (Heartbleed), pivots to application-layer compromise (Web Attack), establishes an internal foothold and turns it into a botnet (Infiltration, Bot), and culminates in availability disruption (DoS, DDoS).

The three anomaly detectors are trained once on the same Normal pool that the continual learners update on and are evaluated on the identical cumulative per-stage test sets.

5. Evaluation

This section reports the empirical evaluation of the continual learning strategies introduced in Section 3. The presentation is organized around questions a practitioner faces when selecting a CL strategy for an adaptive NIDS—which family of methods retains performance under incremental attack arrival Sections 5.3 and 5.4, how that ranking changes with the base architecture Section 5.5, what the picture looks like under class imbalance Section 5.6, how supervised CL compares to an unsupervised anomaly-detection alternative Section 5.7, what it costs in compute and memory Section 5.8, and how robust the answers are to hyperparameters not fixed by the experimental design Section 5.9.

5.1. Setup recapitulation

Each of the three datasets (UQ-IoT-IDS-2021, Kitsune, CIC-IDS-2017; described in Section 4) is split into nine sequential stages following the Cyber-Kill-Chain ordering listed in Section 4.4. At every stage t , the model is updated on the sliding training window D_t and evaluated on the cumulative test set $D_{1:t}^{\text{test}}$. The neural models (MLP, LSTM, Transformer) are trained at every stage on D_t under each CL strategy: WI, three regularization variants (EWC, LwF, SI), three replay variants (ER, A-GEM,

DER + +), and the combined EWC + ER. The baselines RF and SVM are retrained from scratch on D_t at every stage with no CL strategy, serving as a lower reference. The Joint configuration retrains a fresh neural model from scratch on the cumulative dataset $D_{1:t}$ at every stage, serving as an upper bound. As motivated in Section 3, the detection objective is binary (Normal vs. Attack) under a DIL formulation, so the label space is fixed across stages and only the within-class input distribution shifts as new attack types are introduced; per-attack F1 reported below is therefore the binary detector's detection performance restricted to samples carrying that ground-truth attack type. Two evaluation regimes are applied as described in Section 3.1: the *balanced* regime that caps each attack type at the same per-stage size to isolate forgetting from class imbalance and the *unbalanced* regime that applies tier-based caps to retain a normal-dominated, attack-rare class distribution.

5.2. Evaluation metrics

Intrusion detection performance is quantified using the F1-score to account for class imbalance. Let TP , FP , and FN denote the number of true positives, false positives, and false negatives, respectively, where the positive class corresponds to malicious traffic. Precision, recall, and F1-score are defined as:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2 \text{ Precision Recall}}{\text{Precision} + \text{Recall}}.$$

Throughout the paper, TP , FP , and FN are aggregated in two complementary ways: per attack type, restricting the evaluation pool to the samples of that attack type (used for per-attack F1 and the macro-F1 average defined below), and globally over the full cumulative test set with the positive class set to any attack (used for the precision-recall analyses, PR-AUC and FPR at recall ≥ 0.95 , and the binary detection F1 of Section 5.7).

To assess how detection quality varies across the different attack types observed along the continual-learning stream, we report a macro-averaged F1: the unweighted mean, over the attack types introduced so far, of each type's per-attack F1. Because the evaluation pool for attack type A_i contains only A_i traffic, this per-attack F1 measures the fraction of A_i the model still detects, and macro-averaging weights every attack type equally regardless of its prevalence, so that forgetting even a rare early attack is visible.

We further report PR-AUC—the area under the precision-recall curve, which summarizes detector performance across all decision thresholds—FPR at recall ≥ 0.95 , training-time overhead ratio, peak CL-state memory, and an average forgetting rate F on macro-F1. Following Chaudhry et al. [40], F is defined as

$$F = \frac{1}{T-1} \sum_{i=1}^{T-1} \left(\max_{t \in [i, T-1]} f_{t,i} - f_{T,i} \right),$$

where $f_{t,i}$ is macro-F1 on stage i 's test partition (Normal together with the attack types introduced through stage i) evaluated after training through stage t , and $T = 9$. We compute F on macro-F1 rather than accuracy because the IDS classes are imbalanced; accuracy hides minority-class collapse. The training-time overhead ratio is defined as the total wall-clock time required by a CL strategy—comprising both per-stage training and any auxiliary computations such as Fisher information estimation, LwF snapshot retention, and SI consolidation—normalized by the corresponding wall-clock time of WI on the same architecture.

5.3. Comparative performance of CL strategies across datasets

Fig. 9 reports macro-F1 per (strategy, stage) for the MLP architecture across the three datasets, with the classical RF and SVM baselines added as non-CL references; Table 7 consolidates the dataset-averaged metrics under the same row layout. The dominant pattern is consistent across UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017: replay-based strategies (ER, DER + +, and the combined EWC + ER) retain final-stage macro-F1 above 0.95 (balanced regime), while non-replay configurations (MLP

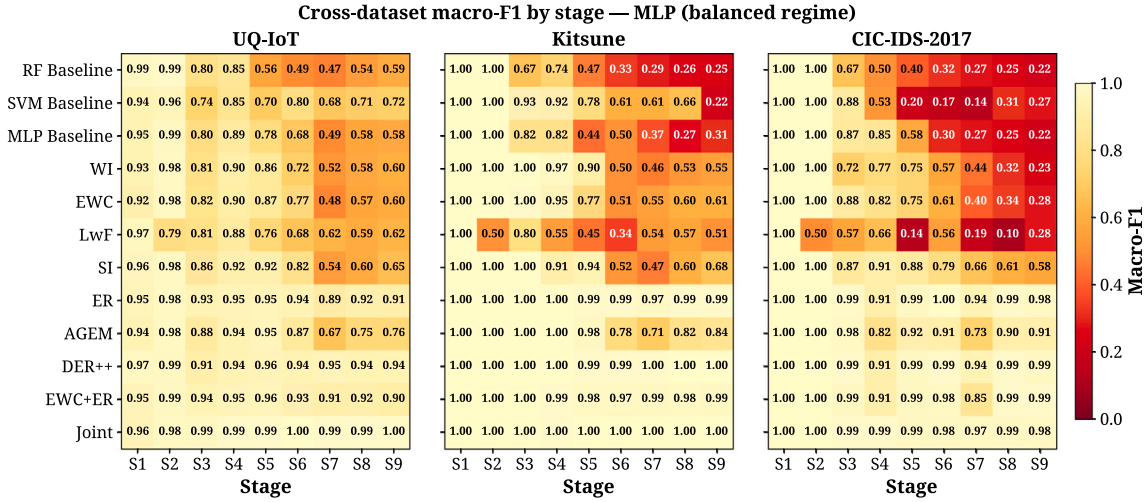


Fig. 9. Per-stage macro-F1 on UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017 (balanced regime). The top three rows report the non-CL references: the classical RF and SVM baselines and the MLP baseline. The remaining rows report MLP under eight continual learning strategies—initialization (WI), regularization (EWC, LwF, SI), replay (ER, AGEM, DER + +), and combined (EWC + ER)—together with the joint-training upper bound (Joint).

Table 7

Average results across UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017. The top block reports MLP under each CL strategy; the bottom block reports the RF and SVM baselines as classical non-CL references. F1 columns report final-stage macro-F1 under balanced and unbalanced regimes; PR-AUC and $FPR_{0.95}$ (FPR proxy at recall ≥ 0.95) are computed on the unbalanced regime; F is the average forgetting rate; “Time $\times$ ” is the train-time overhead ratio relative to WI on the same model; “Mem” is the peak CL-state memory in megabytes. Arrows in the header indicate whether higher ($\uparrow$) or lower ($\downarrow$) is better. The best value per column, excluding baselines and Joint, is shown in bold.

Strategy	F1 (bal) $\uparrow$	F1 (unb) $\uparrow$	PR-AUC $\uparrow$	$FPR_{0.95}$ $\downarrow$	F $\downarrow$	Time $\times$ $\downarrow$	Mem (MB) $\downarrow$
MLP Baseline	0.380 ± 0.016	0.378 ± 0.011	0.839 ± 0.005	0.396 ± 0.013	0.532 ± 0.008	1.03	0.00
WI	0.478 ± 0.024	0.387 ± 0.017	0.829 ± 0.016	0.439 ± 0.024	0.504 ± 0.048	1.00	0.00
EWC	0.491 ± 0.014	0.391 ± 0.013	0.840 ± 0.016	0.420 ± 0.024	0.485 ± 0.035	1.78	1.25
LwF	0.417 ± 0.038	0.368 ± 0.039	0.952 ± 0.006	0.205 ± 0.028	0.432 ± 0.045	1.38	0.63
SI	0.567 ± 0.027	0.566 ± 0.094	0.913 ± 0.019	0.343 ± 0.035	0.429 ± 0.008	1.90	2.50
ER	0.960 ± 0.003	0.932 ± 0.006	0.987 ± 0.000	0.095 ± 0.006	0.040 ± 0.006	1.42	0.32
AGEM	0.828 ± 0.024	0.847 ± 0.037	0.983 ± 0.001	0.129 ± 0.003	0.151 ± 0.020	1.75	0.32
DER + +	0.959 ± 0.008	0.924 ± 0.012	0.986 ± 0.001	0.115 ± 0.015	0.041 ± 0.013	1.45	0.33
EWC + ER	0.956 ± 0.004	0.932 ± 0.007	0.987 ± 0.001	0.093 ± 0.006	0.043 ± 0.003	2.24	1.58
Joint	0.987 ± 0.004	0.943 ± 0.021	0.996 ± 0.000	0.021 ± 0.002	0.004 ± 0.003	1.98	0.00
RF Baseline	0.353 ± 0.007	0.356 ± 0.002	0.942 ± 0.005	0.486 ± 0.000	0.485 ± 0.005	–	0.00
SVM Baseline	0.390 ± 0.008	0.356 ± 0.003	0.875 ± 0.011	0.429 ± 0.016	0.545 ± 0.024	–	0.00

baseline, WI, and the regularization-only variants EWC, LwF, SI) degrade to 0.38–0.57. The classical RF and SVM baselines sit in the same low-retention band (final-stage macro-F1 0.35–0.39 on average). All entries in Table 7 are means over five seeds; standard deviations are small for replay (≤ 0.008 macro-F1) and larger for the seed-sensitive regularization methods (up to 0.038 for LwF).

Three observations are noteworthy. First, no regularization-only strategy closes the gap to the Joint upper bound (final-stage macro-F1 ≈ 0.99 , $F \approx 0.00$). EWC, LwF, and SI all leave the forgetting rate F above 0.31 and final-stage macro-F1 below 0.67, indicating that protecting weights via Fisher information or feature distillation may be insufficient when the input distribution shifts substantially between attack stages. Second, replay alone is sufficient: ER reduces F to 0.04 with a per-class buffer of 100 samples and matches the combined EWC + ER configuration to within 0.004 macro-F1, at substantially lower training-time cost (1.42 $\times$ WI for ER versus 2.24 $\times$ for EWC + ER). Third, WI provides a small but consistent benefit (final-stage macro-F1 +0.10 over MLP Baseline) at negligible time and memory cost, although its effect is dominated by any subsequently added replay or regularization mechanism.

The heatmap also reveals dataset-specific patterns. First, the gap between methods varies sharply by dataset. The spread of final-stage

macro-F1 between the worst and best MLP configuration (excluding the Joint upper bound) is roughly 0.34 on UQ-IoT-IDS-2021 (MLP WI 0.60 vs. MLP DER + + 0.94), but reaches 0.65 on both Kitsune (MLP Baseline 0.35 vs. MLP DER + + 1.00) and CIC-IDS-2017 (MLP Baseline 0.34 vs. MLP EWC + ER 0.99). Second, the non-replay collapse on CIC-IDS-2017 is concentrated at the final two stages: MLP Baseline holds macro-F1 ≥ 0.96 through Stage 7 and then drops to 0.34 by Stage 9, coinciding with the introduction of Bot, DoS, and DDoS, whose high-volume flow features dominate the sliding training window and overwrite the discriminative weights for the earlier reconnaissance and exploitation classes.

5.4. Forgetting and stability

Fig. 10 reports the average forgetting rate F per (strategy, dataset) and, as an inset, the per-stage forgetting matrix $f_{t,i}$ for the representative method MLP ER on UQ-IoT-IDS-2021. The bar pattern mirrors the trends in Fig. 9: replay strategies cluster near $F = 0.05$ (within rounding of the Joint reference), regularization-only methods sit at $F = 0.2$ –0.8, and MLP Baseline together with the classical RF and SVM Baselines exceed $F = 0.29$ on every dataset. CIC-IDS-2017 shows the largest forgetting for non-replay methods ($F \geq 0.78$), an observation we attribute in Section 5.6 to the highly skewed prevalence of the late-stage attacks. The

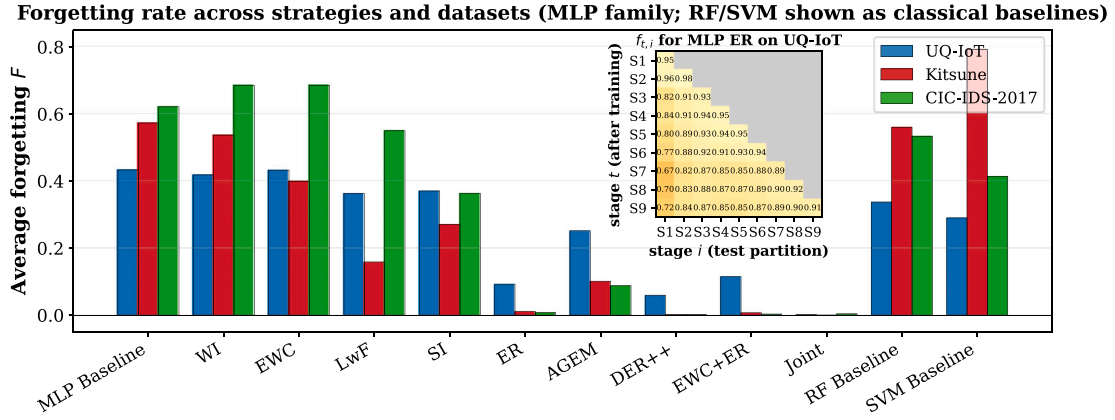


Fig. 10. Average forgetting rate F (Section 5.2) for the MLP family, with RF and SVM baselines shown for context. Inset: the per-stage forgetting matrix $f_{t,i}$ for the representative method MLP ER on UQ-IoT-IDS-2021; cell (t, i) is macro-F1 on stage i 's test partition after training through stage t , with $t < i$ left blank.

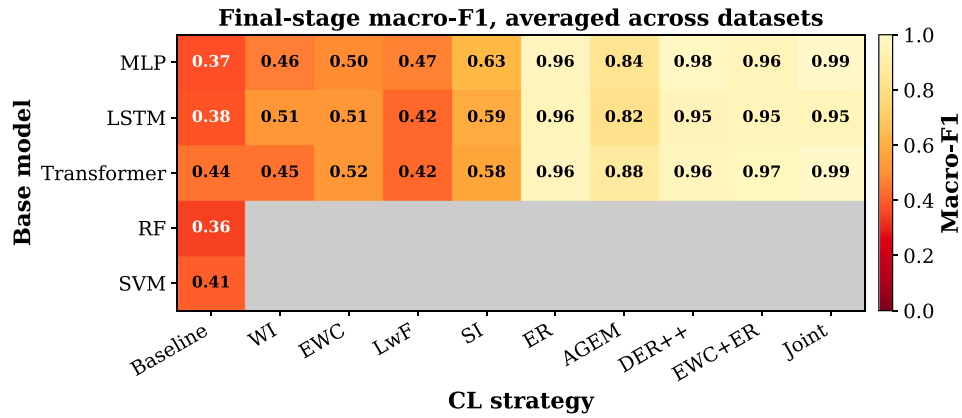


Fig. 11. Final-stage macro-F1 by base model (rows) and CL strategy (columns), averaged across UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017. RF and SVM populate only the Baseline column because they maintain no continual state.

inset reveals *when* forgetting occurs: even for the strongest CL method, the largest dip is at Stage 7 on UQ-IoT—the first stage whose sliding training window contains only flood-based DoS attacks (UDP and SYN Flooding) and therefore offers no direct training signal for the earlier access-layer classes. The just-trained macro-F1 falls from ≈ 0.94 at Stage 6 to 0.89 at Stage 7, and the model partially recovers to ≈ 0.91 at Stages 8–9 thanks to the persistent replay buffer.

5.5. Architecture sensitivity

Fig. 11 reports the final-stage macro-F1 averaged across the three datasets, indexed by base model and CL strategy. The strategy ranking is preserved across MLP, LSTM, and Transformer: replay-based and combined strategies sit at 0.82–0.98 macro-F1 for every deep architecture (with A-GEM at the low end and DER++ / EWC + ER at the high end), while regularization-only methods sit at 0.42–0.63.

Two architecture-specific contrasts stand out among the base models. First, Transformer is the strongest non-CL deep baseline (final-stage macro-F1 0.44, $F = 0.41$), versus 0.37/0.54 for MLP and 0.38/0.50 for LSTM. Second, the benefit of warm initialization (WI) varies substantially by architecture: LSTM gains the most (+0.13 macro-F1 over its Baseline), MLP gains +0.09, and Transformer gains only +0.01 because its Baseline is already comparatively strong. The Joint upper bound also exposes a capacity asymmetry: MLP and Transformer reach ≈ 0.99 macro-F1 with full cumulative data, but LSTM plateaus at 0.95, suggesting that the LSTM's architectural limitations are the bottleneck rather than a forgetting issue.

5.6. Imbalanced regime and per-attack behavior

The balanced regime used so far isolates forgetting from class imbalance, but real NIDS deployments encounter the original prevalence. We juxtapose the two regimes at the per-attack level: Fig. 12 reports per-attack F1 at the final stage under the balanced regime, and Fig. 13 reports the same heatmap under the unbalanced regime, in both cases for five representative MLP configurations (Baseline, WI, the strongest regularizer EWC, the strongest replay ER, and the combined EWC + ER).

The two heatmaps reveal a sharper contrast than the aggregated macro-F1 numbers suggest. Under both regimes, MLP Baseline and WI fail to detect essentially any sample of the earliest-introduced minority attack classes by the final stage: per-attack F1 is at or below 0.03 for ARP Spoofing on UQ-IoT-IDS-2021, Heartbleed and Bot on CIC-IDS-2017, and Active Wiretap on Kitsune in either regime. The clearest balanced-vs-unbalanced contrast is on EWC: balanced EWC partially recovers Bot on CIC-IDS-2017 (per-attack F1 0.52), but the same configuration loses the class entirely under the unbalanced regime (0.00), confirming that Fisher-based regularization is fragile when minority classes face order-of-magnitude prevalence pressure.

Replay nearly closes the gap on those same classes in both regimes: ER and EWC + ER reach per-attack F1 above 0.95 on Heartbleed, Bot, and Active Wiretap regardless of the regime. The one exception is ARP Spoofing on UQ-IoT-IDS-2021, where ER reaches 0.79 under the balanced regime but only 0.54 under the unbalanced regime—the only minority class on which replay's own performance degrades visibly between the two regimes. The combined EWC + ER stays within 0.03 of

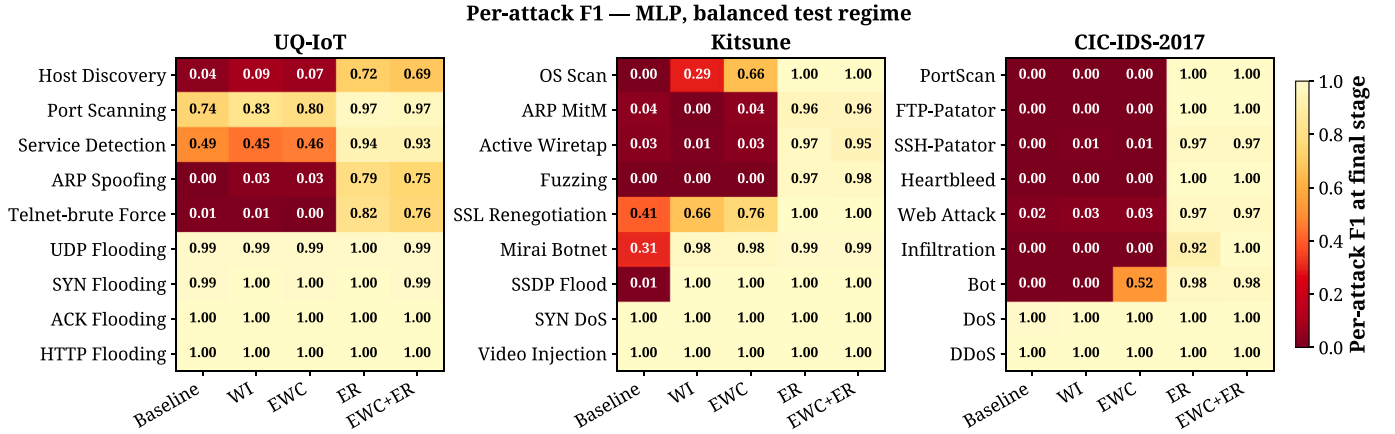


Fig. 12. Per-attack F1 at the final stage in the *balanced* regime for five representative MLP configurations. Rows list dataset-specific attack classes in the order.

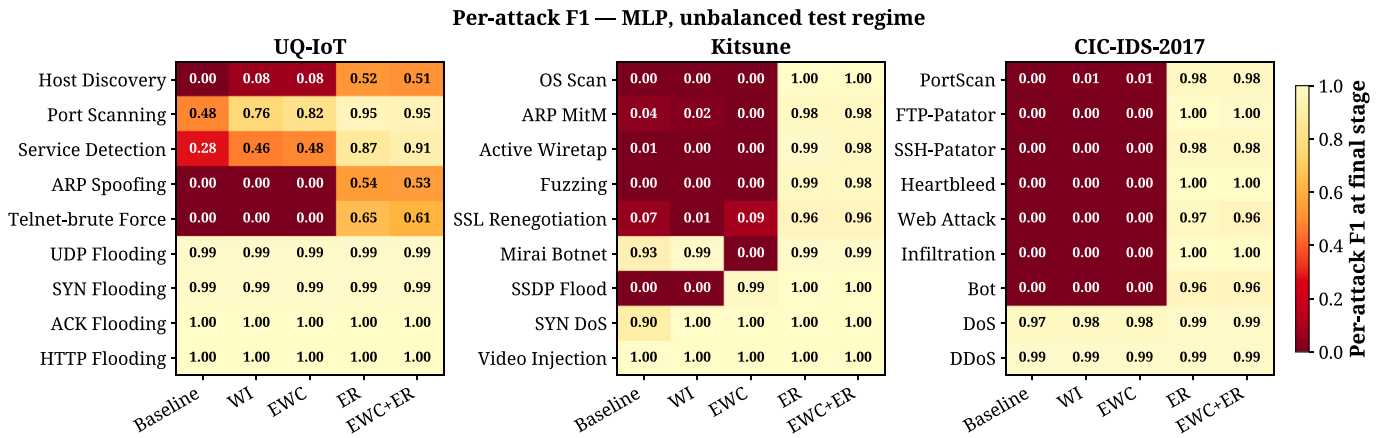


Fig. 13. Per-attack F1 at the final stage in the *unbalanced* regime for the same five MLP configurations as Fig. 12.

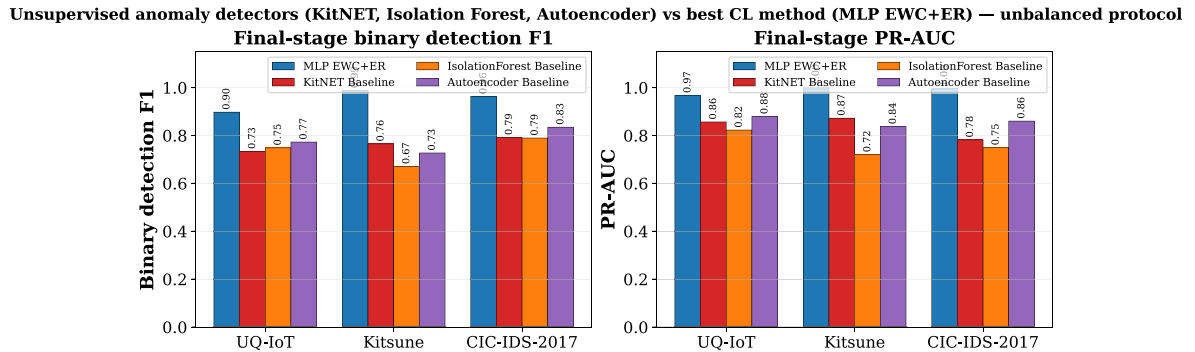


Fig. 14. Three unsupervised anomaly detectors (KitNET, Isolation Forest, autoencoder) vs the best CL method (MLP EWC + ER) on the final stage of each dataset, unbalanced protocol. Binary detection F1 (left; attack classes pooled against Normal, so false positives on Normal are counted) and threshold-free PR-AUC (right). Each detector is fitted on the same train-fold Normal pool as the CL models, with its threshold and hyperparameters selected on a labeled validation slice.

ER under both regimes, aligning with the finding in Section 5.3 that EWC adds little once replay is in place.

5.7. Comparison with anomaly-detection baselines

A single fixed anomaly detector is a narrow reference, so we compare the strongest CL configuration (MLP EWC + ER) against three unsupervised detectors: KitNET [9], an Isolation Forest, and an autoencoder. Each is fitted on the same Normal pool as the continual

learners, with its threshold and hyperparameters tuned on a labeled validation slice spanning all attack families (Section 4.3), a setup that favors the detectors. Fig. 14 reports final-stage binary detection F1 (attacks pooled against Normal, so false positives on Normal count) and threshold-free PR-AUC under the unbalanced regime. MLP EWC + ER leads every detector on every dataset, by 0.13–0.32 in binary detection F1 and 0.09–0.28 in PR-AUC across the nine detector–dataset combinations. The gap is driven by precision: at their F1-optimal thresholds the

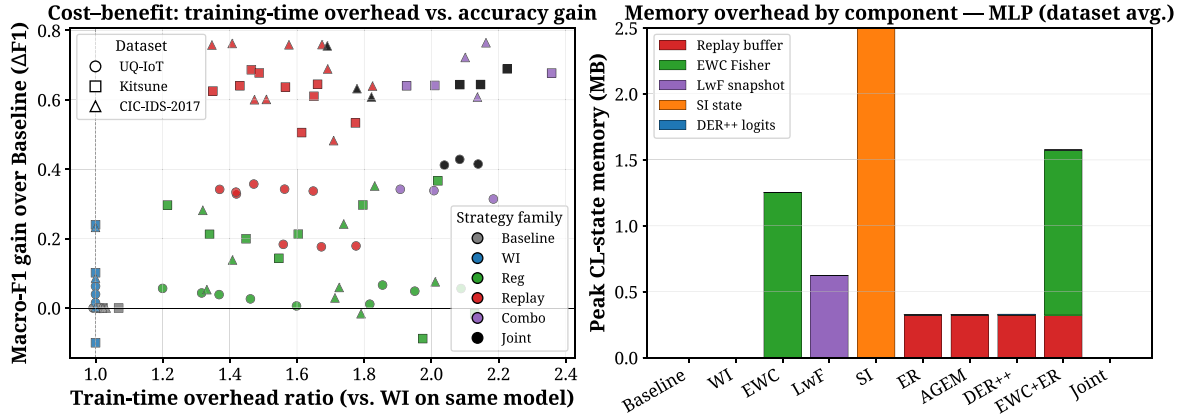


Fig. 15. Resource overhead. Left: training-time overhead ratio (relative to WI on the same model) versus macro-F1 gain over MLP baseline; marker shape encodes dataset and color encodes strategy family. Right: peak CL-state memory split by component (replay buffer, EWC Fisher, LwF snapshot, SI state, DER++ logits) for the MLP architecture, averaged across datasets.

detectors flag a large fraction of Normal traffic, holding their precision to 0.51–0.88 against 0.96–1.00 for MLP EWC + ER. It is most pronounced for the Isolation Forest on Kitsune, which reaches near-complete attack recall by flagging 99.6% of Normal traffic, for a binary detection F1 of 0.67. Because the ranking also holds under PR-AUC, the deficit reflects weaker separation of attack from normal scores, not an unlucky operating point.

The detectors model only Normal traffic and never train on attack samples. Because even label-informed threshold tuning leaves a wide gap, the comparison isolates the value of discriminative training on labeled attacks rather than the value of labels for calibration alone. Re-fitting a detector per stage would not close the gap: the Normal distribution is already fully available at Stage 1, so the deficit stems from fitting on Normal alone, not from training “once and fixed.” Threshold-based anomaly detection therefore remains appropriate mainly for cold-start settings where labeled attack samples are unavailable.

5.8. Resource overhead

Fig. 15 reports the training-time overhead ratio against the macro-F1 gain over the MLP Baseline (left), and the peak CL-state memory split by component for the MLP family (right). Replay-based strategies sit in the upper portion of the cost-benefit plot—they yield large macro-F1 gains (0.18–0.77 over the MLP Baseline depending on the dataset and replay variant) at modest training-time cost (1.3–1.9× WI). Regularization-only methods sit in the lower portion (small gain at comparable cost) because the Fisher and SI consolidation steps are non-trivial relative to their benefit. Regularization-only methods sit in the lower portion: their per-stage Fisher (EWC) and path-integral consolidation (SI) steps cost roughly as much training time as replay, but deliver a fraction of the macro-F1 gain. The peak CL-state memory of ER with a 100-sample-per-class replay buffer is approximately 0.32 MB on the MLP architecture—about half the model’s own parameter footprint (0.63 MB) and many times smaller than the cumulative training data on disk—indicating that the resource cost of memory-based CL is compatible with the edge-deployment budget. The combined EWC + ER incurs 1.25 MB of additional state and 0.8× extra training time over ER without yielding significant improvement.

5.9. Sensitivity analysis

We confirm robustness to three hyperparameters that are not fixed by the experimental design—the regularization strength λ for each regularizer, the replay buffer size, and the order in which attack classes are introduced—and report each sweep across all three datasets on the MLP backbone under the balanced regime.

5.9.1. Regularization strength

Fig. 16 sweeps the regularizer coefficient for each method that uses one. EWC is essentially insensitive to $\lambda \in \{0.2, 0.4, 0.8\}$: intra-dataset variation is at most 0.06 macro-F1. SI ($\lambda \in \{0.5, 1.0, 2.0\}$) is moderately variable: the best λ shifts between 0.5 and 2.0 across datasets and intra-dataset spreads reach 0.14 macro-F1. LwF is the most sensitive of the three. Its best λ differs by dataset (0.5 on UQ-IoT-IDS-2021 and CIC-IDS-2017, 1.0 on Kitsune), and on the Kitsune and CIC-IDS-2017 datasets macro-F1 collapses by 0.22–0.24 between the best and worst grid values, so LwF requires per-dataset tuning to avoid suppressing the new-task signal under strong distillation.

5.9.2. Replay buffer size

Fig. 17 compares per-class buffer sizes 10, 30, 100, and 300 for the four replay-based methods (ER, EWC + ER, A-GEM, DER++). On Kitsune and CIC-IDS-2017, ER, DER++, and EWC + ER all reach final-stage macro-F1 ≥ 0.95 with as few as 10 samples per class—the per-class feature distributions are tight enough that a small buffer slice already covers them. On UQ-IoT-IDS-2021 the curve is steeper: ER moves from 0.72 (buf = 10) to 0.95 (buf = 300), with the largest single jump between 10 and 30 samples per class. The gain from 100 to 300 samples on UQ-IoT-IDS-2021 is approximately 0.04 macro-F1 at 3× the buffer memory; on the other two datasets it is below 0.01. We adopt 100 samples per class throughout the headline experiments. A-GEM is the one outlier: its final-stage macro-F1 is non-monotonic in buffer size (e.g., on CIC-IDS-2017 it decreases from 0.89 at buf = 10 to 0.71 at buf = 100), indicating that it is sensitive to which samples sit in the buffer rather than to how many.

5.9.3. Task order

A single alternative ordering cannot characterize order sensitivity, so we evaluate each MLP configuration on *five seeded random attack orders* per dataset (seeds 0–4), alongside the default Cyber-Kill-Chain and flooding-first orderings as references. Fig. 18 reports the default-order final-stage macro-F1 and the mean $\pm$ standard deviation across the five random orders. Replay-based methods are highly order-robust—across-order standard deviation ≤ 0.026 and worst-case spread ≤ 0.05 macro-F1 (ER/EWC + ER stay at 0.93–0.99 on every dataset)—whereas non-replay methods are far more sensitive, with across-order standard deviations up to 0.13 and spreads up to 0.30 (e.g., WI 0.324 ± 0.127 on CIC-IDS-2017). Averaged over all methods and datasets, the across-order standard deviation is 0.017 for replay versus 0.097 for non-replay—roughly six-fold. Because the emergence order of new attacks is uncontrollable in deployment, replay-based methods are the safer default: not strictly order-invariant, but substantially more order-robust.

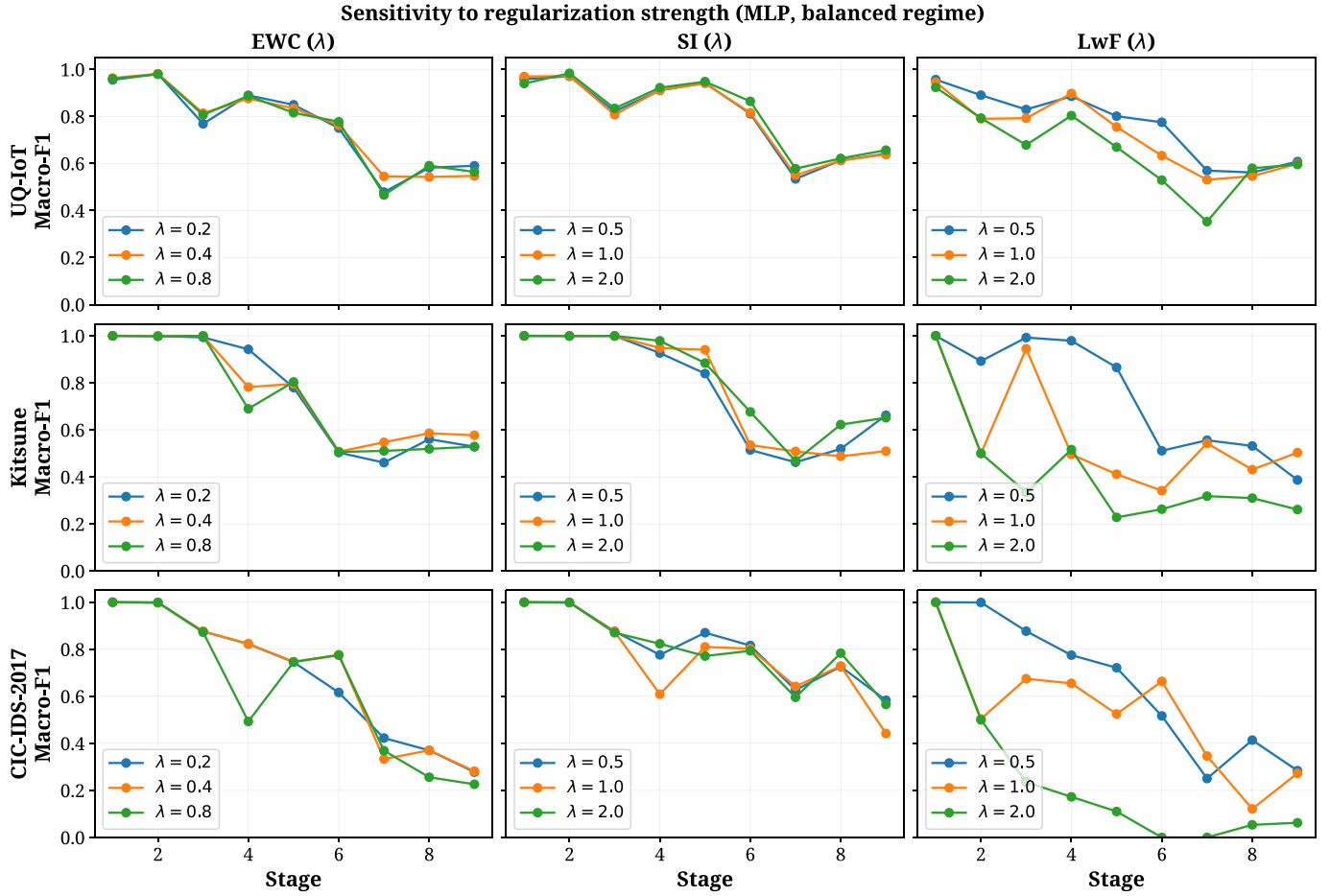


Fig. 16. Sensitivity to regularization strength: per-stage macro-F1 of MLP EWC, MLP SI, and MLP LwF under each method's λ grid, on UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017 (balanced regime).

6. Discussion

Summary of Findings. The evaluation supports six claims that hold across three datasets, three neural architectures, and the unbalanced regime: (i) replay is the dominant lever for mitigating catastrophic forgetting in NIDS—without it, average forgetting F stays above 0.32 even for the strongest regularizer; (ii) regularization alone (EWC, LwF, SI) provides no consistent improvement over WI, and combining regularization with replay yields no measurable gain over replay alone (Table 7); (iii) the strategy ranking transfers across MLP, LSTM, and Transformer architectures (Section 5.5); (iv) replay's resource footprint is modest—0.32 MB CL state and $1.4\times$ WI training time (Section 5.8); (v) supervised continual learning outperforms three unsupervised anomaly-detection baselines (KitNET, Isolation Forest, and an autoencoder) by 0.13–0.32 binary detection F1 across the three datasets, isolating the value of discriminative training on labeled attacks (Section 5.7); and (vi) replay-based methods are substantially more robust to the order in which new attacks are introduced than non-replay methods: across five seeded random task orders per dataset, replay's final-stage macro-F1 varies with an across-order standard deviation ≤ 0.026 (worst-case spread ≤ 0.05), whereas non-replay methods reach across-order standard deviations up to 0.13 (worst-case spread up to 0.30) (Section 5.9).

Overall Comparison of CL Strategies. A consistent performance gap separates the evaluated CL strategies in the NIDS setting. Across the nine sequential stages, replay-based configurations (ER, DER + +, EWC + ER) retain final-stage macro-F1 above 0.92 on every dataset, while WI and the regularization-only variants (EWC, LwF, SI) degrade progressively to 0.38–0.57 (Table 7). The gap is consistent across MLP,

LSTM, and Transformer base models (Fig. 11), indicating that the choice of CL mechanism is more decisive than the choice of architecture for sustaining detection performance under stage-wise distribution shifts.

Effectiveness of Replay in DIL Settings. Replay sidesteps the structural mismatch by reintroducing historical samples directly during training, thereby approximating joint optimization over past and current data under a constrained memory budget. This mechanism aligns naturally with the DIL formulation in Section 3.2, in which the detection objective is fixed and only the within-attack input marginal $P(X | Y = \text{Attack})$ shifts. The advantage is most pronounced precisely where regularization fails: on rare attack classes such as Heartbleed, Bot, and Active Wiretap, ER and EWC + ER maintain per-attack F1 above 0.95 under both balanced and unbalanced regimes (Section 5.6), whereas non-replay strategies collapse to $F1 \leq 0.03$ on the same classes. This pattern is consistent with concurrent NIDS-specific findings on adaptive memory replay under data drift [23].

Robustness of the Strategy Ranking. Beyond its consistency across the three architectures (Section 5.5), the strategy ranking is robust to task order, a factor practitioners cannot control a priori: across five seeded random task orderings per dataset (Section 5.9), replay-based methods remain far more stable than non-replay strategies, so the operationally uncontrollable emergence order of new attacks does not upset the replay advantage. The architecture comparison also surfaces a secondary observation: the LSTM upper bound plateaus at 0.95 macro-F1 even under joint training, locating the LSTM's limitation in its representational capacity rather than in catastrophic forgetting. For latency-sensitive deployments, MLP + ER is therefore a reasonable default.

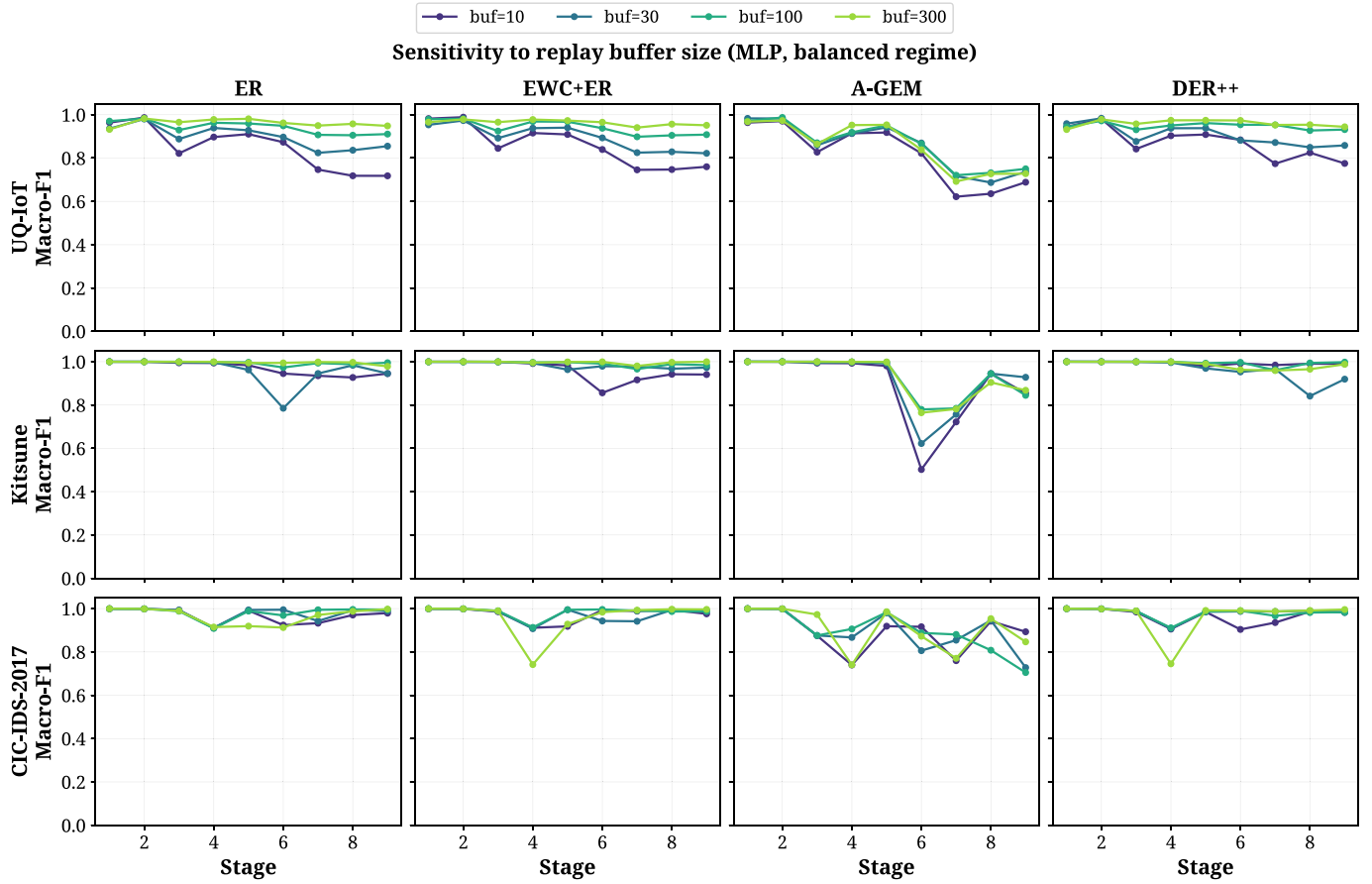


Fig. 17. Sensitivity to replay buffer size: per-stage macro-F1 of the four replay-based MLP configurations (ER, EWC + ER, A-GEM, DER + +; columns) at four buffer sizes per class (10, 30, 100, 300), on UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017 (rows; balanced regime).

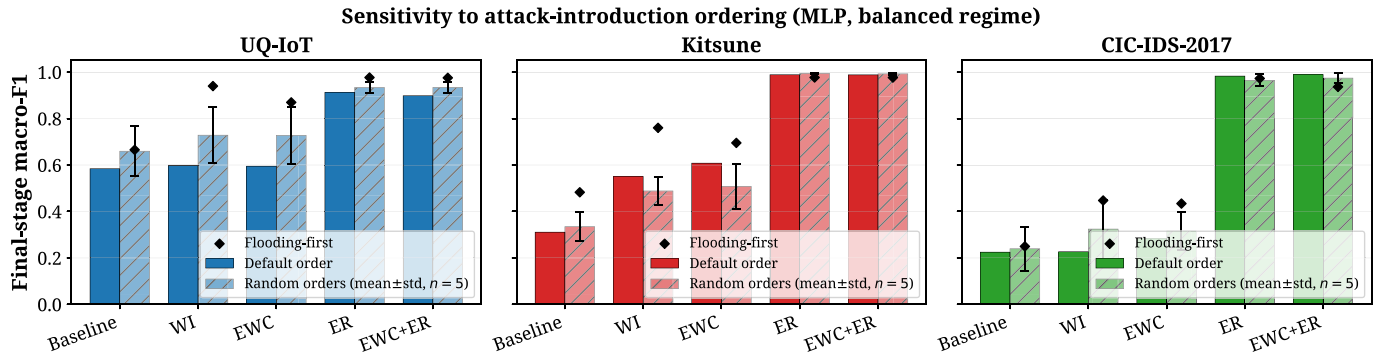


Fig. 18. Sensitivity to attack-introduction ordering: final-stage macro-F1 of five MLP configurations under the default Cyber-Kill-Chain ordering (solid bars) and the mean $\pm$ standard deviation across five seeded random orders (hatched bars; error bars denote the standard deviation across orders), with the designed flooding-first order overlaid as a marker, on each dataset (balanced regime). Replay-based configurations (ER, EWC + ER) are substantially more order-robust than the non-replay configurations (Baseline, WI, EWC).

Cost-Benefit and the Choice between Supervised CL and Anomaly Detection. The replay configuration that delivers the strongest detection performance is also one of the cheapest. The peak CL state for ER on MLP is about half the model's own parameter footprint, and its training-time overhead is below that of every regularization-only alternative (Section 5.8); a per-class buffer of 100 raw samples is already on the saturation plateau of the retention curve (Section 5.9). Set against this, the three unsupervised anomaly detectors (Section 5.7), trained on the same normal-sample budget, all trail the strongest CL configuration by a substantial margin, widest on Kitsune and comparable on

UQ-IoT-IDS-2021 and CIC-IDS-2017. The combined picture is that supervised CL is both more accurate and resource-light when labeled samples of newly emerging attack classes are available; threshold-based anomaly detection should be reserved for cold-start scenarios in which such labels cannot be obtained.

Practical Limits of Replay in IoT Deployment. Three practical considerations qualify the deployment of replay-based CL on IoT NIDS. *Memory and compute* are quantified above and remain compatible with edge-device budgets. *Privacy* is a more substantive limitation: because the buffer stores raw flow-level feature vectors rather than embeddings

or synthetic representations, retained samples may remain linkable to specific devices, sessions, or users in operational telemetry. The present framework does not implement privacy-preserving replay [41]. Several alternative replay designs—such as compressed-feature storage [42] or federated continual learning across edge nodes [43,44]—could in principle reduce this exposure, although each is likely to involve distinct utility–privacy trade-offs whose comparative evaluation lies beyond the scope of this work. *Concept drift in benign traffic* is a related caveat: the DIL formulation in Section 3.2 assumes that $P_i(X | Y = \text{Normal})$ is approximately stable across stages, but in real deployments this assumption can be violated by firmware updates, the introduction of new device classes, or seasonal usage patterns [16]. Where the assumption is violated, the buffer's Normal slot itself becomes stale and must be refreshed; otherwise the model preserves an outdated benign decision boundary.

Limitations and Future Directions. Beyond the practical limits above, three further directions emerge from the experimental scope. First, although the three evaluated datasets span IoT-specific and enterprise traffic with diverse acquisition environments, broader benchmarking on emerging datasets such as TON IoT or IoT 23 would strengthen the external validity of the conclusions. Second, the binary detection formulation isolates retention behavior under domain shift; extending the framework to a class-incremental multi-class setting in which attack-type attribution is performed jointly is a natural next step toward operational forensics. Third, the staged introduction of attack classes is a controlled approximation of non-stationary threat environments; in practice, attack behaviors may evolve in more complex ways, including gradual drift and concurrent emergence of multiple attack families, neither of which is exercised here.

7. Conclusions

We have investigated CL strategies for IoT NIDS under a DIL setting, where the detection objective remains fixed while attack distributions evolve over time. To this end, we have designed a CL evaluation framework based on an incremental training and cumulative testing protocol that reflects how an NIDS is practically updated as new attack types emerge. Using three real-world datasets (UQ-IoT-IDS-2021, Kitsune, and CIC-IDS-2017) and three deep learning architectures (MLP, LSTM, Transformer), we have conducted a systematic evaluation of eight CL strategies, classical baselines, and an anomaly-based reference, focusing on their ability to retain detection performance across sequential stages under both balanced and unbalanced protocols.

Across nine sequential task scenarios on every dataset, the experimental results have shown that replay-based learning is particularly effective in mitigating catastrophic forgetting. Specifically, ER incorporated within a WI-based framework retains macro-F1 above 0.92 with average forgetting below 0.05, using a modest replay buffer (e.g., 100 samples per class) and a peak CL-state memory under 0.4 MB. In contrast, WI alone, EWC, LwF, and SI each leave forgetting above 0.32 and final-stage macro-F1 below 0.67. Further analyses on the replay buffer size, the regularization strength λ , and the task order indicate that replay-based methods provide stable retention across a range of configurations, while the benefits of regularization alone remain limited in this setting. The observed strategy ranking is consistent across MLP, LSTM, and Transformer, indicating that the conclusions are robust to architectures. In addition, comparison with three unsupervised anomaly-based baselines (KiNET, Isolation Forest, and an autoencoder), trained on a matched normal-sample budget, suggests that supervised continual learning is a more effective choice when labeled samples of new attack classes become available. Overall, these findings suggest the importance of memory-based mechanisms and controlled evaluation protocols for understanding and improving CL behavior in adaptive IoT NIDS.

CRedit authorship contribution statement

Oliver Delgado: Writing – original draft, Software, Methodology, Investigation, Data curation, Conceptualization. **Hyunjae Kang:** Writing – review & editing, Visualization, Validation, Software, Methodology. **Ulysses Lam:** Writing – review & editing, Visualization, Validation, Data curation. **Jung Taek Seo:** Supervision, Conceptualization. **Dan Dongseong Kim:** Writing – review & editing, Supervision, Project administration, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Korea Institute of Energy Technology Evaluation and Planning (KETEP) grant funded by the Korean government (MOTIE) (A Study on the Development of a Cyber-Physical Attack Response System and Security Management System for Maximizing the Availability of Real-Time Distributed Resources, RS-2023-00303559).

Data availability

Data will be made available upon request.

References

- [1] S. Sinha, State of IoT 2024: Number of Connected IoT Devices Growing 13% to 18.8 Billion Globally, IoT Analytics (May 2024). <https://iot-analytics.com/state-of-iot-2024-number-connected-iot-devices-19-billion/>. Market Report.
- [2] S. Yang, X. Zheng, J. Li, J. Xu, X. Wang, E.C.H. Ngai, ReCDA: concept drift adaptation with representation enhancement for network intrusion detection, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, 2024, pp. 3818–3828, <https://doi.org/10.1145/3637528.3672007>.
- [3] F. Jemili, K. Jouini, O. Korbaa, Intrusion detection based on concept drift detection and online incremental learning, Int. J. Pervasive Comput. Commun. 21 (1) (2024) 81–115, <https://doi.org/10.1108/IJPPCC-12-2023-0358>.
- [4] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A.A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al., Overcoming catastrophic forgetting in neural networks, Proc. Natl. Acad. Sci. 114 (13) (2017) 3521–3526, <https://doi.org/10.1073/pnas.1611835114>.
- [5] G.M. van de Ven, A.S. Tolias, Three scenarios for continual learning, 2019, <https://arxiv.org/abs/1904.07734>.
- [6] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, T. Tuytelaars, A continual learning survey: defying forgetting in classification tasks, IEEE Trans. Pattern Anal. Mach. Intell. 44 (7) (2022) 3366–3385, <https://doi.org/10.1109/TPAMI.2021.3057446>.
- [7] L. Wang, X. Zhang, H. Su, J. Zhu, A comprehensive survey of continual learning: theory, method and application, IEEE Trans. Pattern Anal. Mach. Intell. 46 (8) (2024) 5362–5383, <https://doi.org/10.1109/TPAMI.2024.3367329>.
- [8] K. He, D. Kim, Z. Zhang, M. Ge, U. Lam, J. Yu, UQ IoT IDS dataset 2021, 2022, <https://doi.org/10.48610/17b44bb>.
- [9] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: an ensemble of autoencoders for online network intrusion detection, in: Proceedings 2018 Network and Distributed System Security Symposium, Internet Society, San Diego, CA, 2018, <https://doi.org/10.14722/ndss.2018.23211>.
- [10] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116, <https://doi.org/10.5220/0006639801080116>.
- [11] F. Zenke, B. Poole, S. Ganguli, Continual learning through synaptic intelligence, in: D. Precup, Y.W. Teh (Eds.), Proceedings of the 34th International Conference on Machine Learning, 70 Proceedings of Machine Learning Research, PMLR, 2017, pp. 3987–3995.
- [12] Z. Li, D. Hoiem, Learning without forgetting, IEEE Trans. Pattern Anal. Mach. Intell. 40 (12) (2018) 2935–2947, <https://doi.org/10.1109/TPAMI.2017.2773081>.
- [13] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, G. Wayne, Experience replay for continual learning, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, R. Garnett (Eds.), Advances in Neural Information Processing Systems, 32, Curran Associates, Inc., 2019.
- [14] A. Chaudhry, M. Ranzato, M. Rohrbach, M. Elhoseiny, Efficient lifelong learning with A-GEM, in: International Conference on Learning Representations, 2019.

- [15] P. Buzzega, M. Boschini, A. Porrello, D. Abati, S. CALDERARA, Dark experience for general continual learning: a strong, simple baseline, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, 33, Curran Associates, Inc., 2020, pp. 15920–15930.
- [16] J. Kozal, J. Zwolińska, M. Klonowski, M. Woźniak, Defending network IDS against adversarial examples with continual learning, in: 2023 IEEE International Conference on Data Mining Workshops (ICDMW), 2023, pp. 60–69, <https://doi.org/10.1109/ICDMW60847.2023.00017>
- [17] H. Kim, D.D. Kim, J.-H. Cho, T.J. Moore, F.F. Nelson, H. Lim, Continual learning with network intrusion dataset, in: 2022 IEEE International Conference on Big Data (Big Data), 2022, pp. 6702–6704, <https://doi.org/10.1109/BigData55660.2022.10020865>
- [18] S.K. Amalapuram, A. Tadwai, R. Vinta, S.S. Channappayya, B.R. Tamma, Continual learning for anomaly based network intrusion detection, in: 2022 14th International Conference on COMMunication Systems & NETWORKS (COMSNETS), 2022, pp. 497–505, <https://doi.org/10.1109/COMSNETS53615.2022.9668482>
- [19] M.R. Martina, G.L. Foresti, A continuous learning approach for real-time network intrusion detection, *Int. J. Neural Syst.* 31 (12) (2021) 2150060, <https://doi.org/10.1142/S012906572150060X>
- [20] H. Batra, R. Clark, EVCL: elastic variational continual learning with weight consolidation, *arXiv preprint ICML 2024 Workshop on Structured Probabilistic Inference & Generative Modeling*, 2024 [arXiv:2406.15972](https://arxiv.org/abs/2406.15972).
- [21] T. Althiyabi, I. Ahmad, M.O. Allassafi, Enhancing IoT security: a few-shot learning approach for intrusion detection, *Mathematics* 12 (7) (2024) 1055, <https://doi.org/10.3390/math12071055>
- [22] S.K. Amalapuram, S. Channappayya, B.R. Tamma, Augmented memory replay-based continual learning approaches for network intrusion detection, *Adv. Neural Inf. Process. Syst.* 36 (2023) 17156–17169.
- [23] A.H. Nasreen Fathima, A. Khraisat, S.P. Syed Ibrahim, G. Li, Adaptive memory replay for network intrusion detection: tackling data drift and catastrophic forgetting, *Comput. Netw.* 272 (2025) 111712, <https://doi.org/10.1016/j.comnet.2025.111712>
- [24] S.K. Amalapuram, B.R. Tamma, S.S. Channappayya, SPIDER: a semi-supervised continual learning-based network intrusion detection system, in: *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 571–580, <https://doi.org/10.1109/INFOCOM52122.2024.10621428>
- [25] S. Prasath, K. Sethi, D. Mohanty, P. Bera, S.R. Samantaray, Analysis of continual learning models for intrusion detection system, *IEEE Access* 10 (2022) 121444–121464, <https://doi.org/10.1109/ACCESS.2022.3222715>
- [26] C. Oikonomou, I. Iliopoulos, D. Ioannidis, D. Tzovaras, A multi-class intrusion detection system based on continual learning, in: 2023 IEEE International Conference on Cyber Security and Resilience (CSR), 2023, pp. 86–91, <https://doi.org/10.1109/CSR57506.2023.10224974>
- [27] X. Zhang, R. Zhao, Z. Jiang, H. Chen, Y. Ding, E.C.H. Ngai, S.-H. Yang, Continual learning with strategic selection and forgetting for network intrusion detection, in: *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, <https://doi.org/10.1109/INFOCOM55648.2025.11044615>
- [28] X. Zhang, R. Zhao, Z. Jiang, Z. Sun, Y. Ding, E.C. Ngai, S.-H. Yang, AOC-IDS: autonomous online framework with contrastive learning for intrusion detection, in: *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 581–590, <https://doi.org/10.1109/INFOCOM52122.2024.10621346>
- [29] S.K. Amalapuram, T.T. Reddy, S.S. Channappayya, B.R. Tamma, On handling class imbalance in continual learning based network intrusion detection systems, in: *Proceedings of the First International Conference on AI-ML Systems, AIMLSystems '21*, Association for Computing Machinery, New York, NY, USA, 2021, pp. 1–7, <https://doi.org/10.1145/3486001.3486231>.
- [30] X. Tao, X. Hong, X. Chang, S. Dong, X. Wei, Y. Gong, Few-shot class-incremental learning, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 12180–12189, <https://doi.org/10.1109/CVPR42600.2020.01220>
- [31] L. Du, Z. Gu, Y. Wang, L. Wang, Y. Jia, A few-shot class-incremental learning method for network intrusion detection, *IEEE Trans. Netw. Serv. Manag.* 21 (2) (2024) 2389–2401, <https://doi.org/10.1109/TNSM.2023.3332284>
- [32] V. Shanmugam, R. Razavi-Far, E. Hallaji, Addressing class imbalance in intrusion detection: a comprehensive evaluation of machine learning approaches, *Electronics* 14 (1) (2025), <https://doi.org/10.3390/electronics14010069>
- [33] M. Asadi, M.A.J. Jamali, A. Heidari, N.J. Navimipour, Botnets unveiled: a comprehensive survey on evolving threats and defense strategies, *Trans. Emerg. Telecommun. Technol.* 35 (11) (2024) e5056, <https://doi.org/10.1002/ett.5056>
- [34] M.A. Ferrag, L. Maglaras, S. Moschoyiannis, H. Janicke, Deep learning for cyber security intrusion detection: approaches, datasets, and comparative study, *J. Inf. Secur. Appl.* 50 (2020) 102419, <https://doi.org/10.1016/j.jisa.2019.102419>
- [35] C. Guo, X. Li, J. Cheng, S. Yang, H. Gong, Continual learning for intrusion detection under evolving network threats, *Future Internet* 17 (10) (2025), <https://doi.org/10.3390/fi17100456>
- [36] A.H. Lashkari, G.D. Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy - Volume 1: ICISSP, INSTICC, INSTICC*, 2017, pp. 253–262, <https://doi.org/10.5220/0006105602530262>
- [37] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshlein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: an imperative style, high-performance deep learning library, in: *Advances in Neural Information Processing Systems*, 32, Curran Associates, Inc, 2019.
- [38] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-Learn: machine learning in Python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830.
- [39] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, in: *International Conference on Learning Representations (ICLR)*, 2015.
- [40] A. Chaudhry, P.K. Dokania, T. Ajanthan, P.H.S. Torr, Riemannian walk for incremental learning: understanding forgetting and intransigence, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 532–547.
- [41] T. Verma, L. Jin, J. Zhou, J. Huang, M. Tan, B.C.M. Choong, T.F. Tan, F. Gao, X. Xu, D.S. Ting, Y. Liu, Privacy-preserving continual learning methods for medical image classification: a comparative analysis, *Front. Med.* Volume 10 - 2023 (2023), <https://doi.org/10.3389/fmed.2023.1227515>
- [42] L. Pellegrini, G. Graffieti, V. Lomonaco, D. Maltini, Latent replay for real-time continual learning, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020, pp. 10203–10209, <https://doi.org/10.1109/IROS45743.2020.9341460>
- [43] J. Yoon, W. Jeong, G. Lee, E. Yang, S.J. Hwang, Federated continual learning with weighted inter-client transfer, in: M. Meila, T. Zhang (Eds.), *Proceedings of the 38th International Conference on Machine Learning*, 139 *Proceedings of Machine Learning Research*, PMLR, 2021, pp. 12073–12086.
- [44] Z. Jin, J. Zhou, B. Li, X. Wu, C. Duan, FL-IIDS: a novel federated learning-based incremental intrusion detection system, *Future Gener. Comput. Syst.* 151 (2024) 57–70, <https://doi.org/10.1016/j.future.2023.09.019>